Combinatorial Optimization
（組合せ最適化）

Introduction to Big Data Science（ビッグデータ概論）

Ryo Kuroiwa (黒岩稜)

大学共同利用機関法人 情報・システム研究機構
国立情報学研究所
National Institute of Informatics

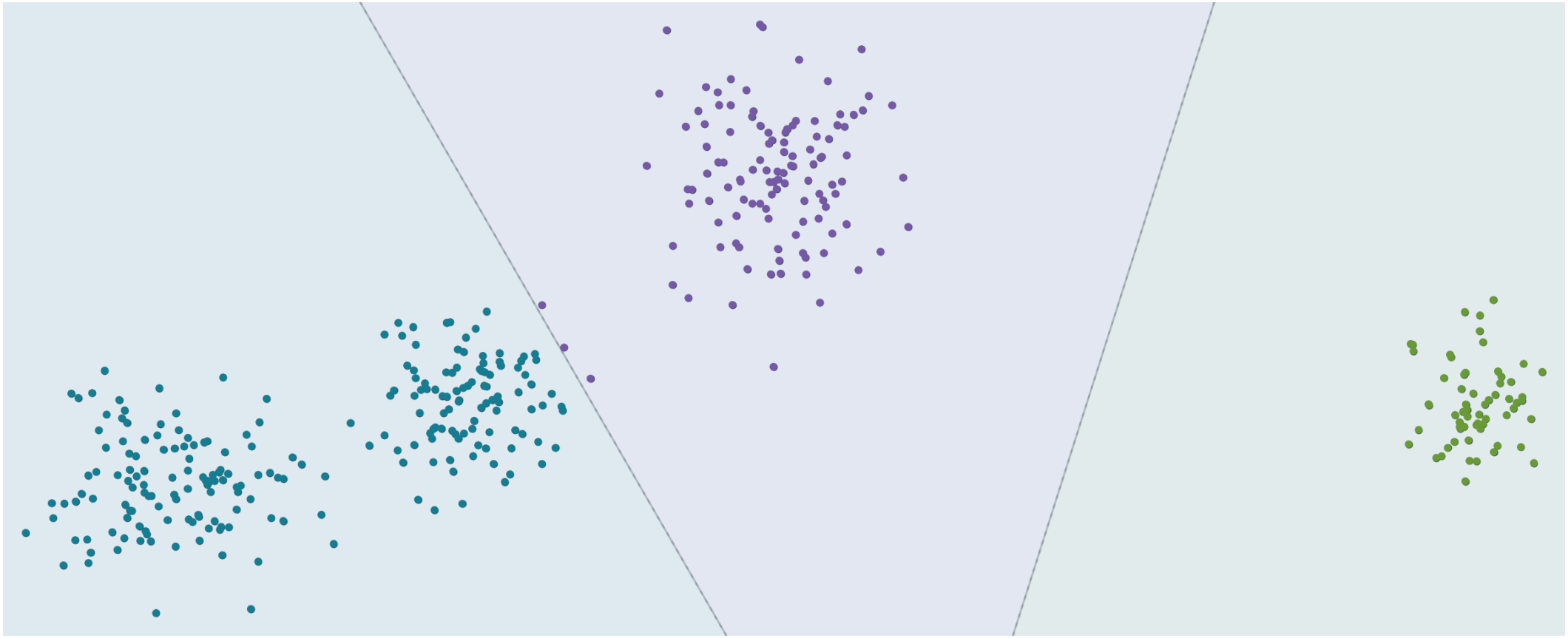
Select Representative Points from Big Data

Example: select customers to interview to understand diverse needs



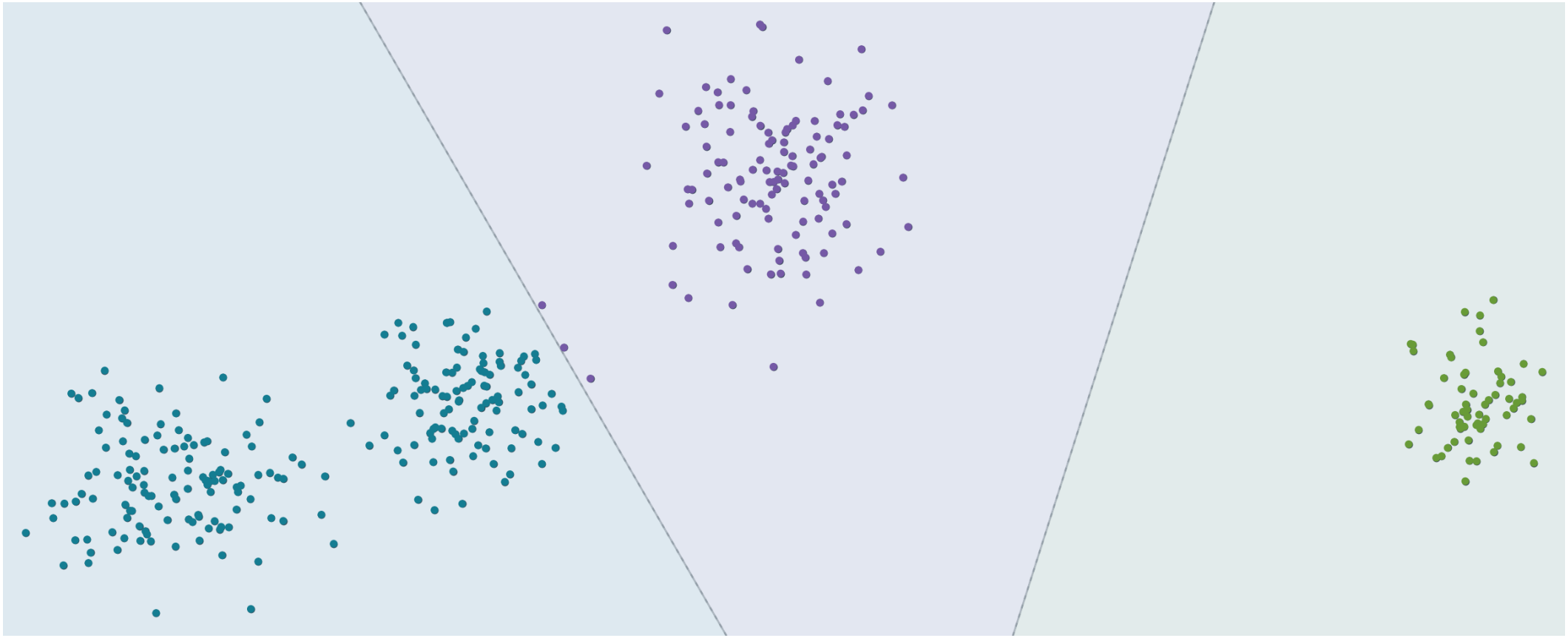
Clustering?

- Partition the data points into clusters, e.g., using k-means clustering
- Select one from each cluster



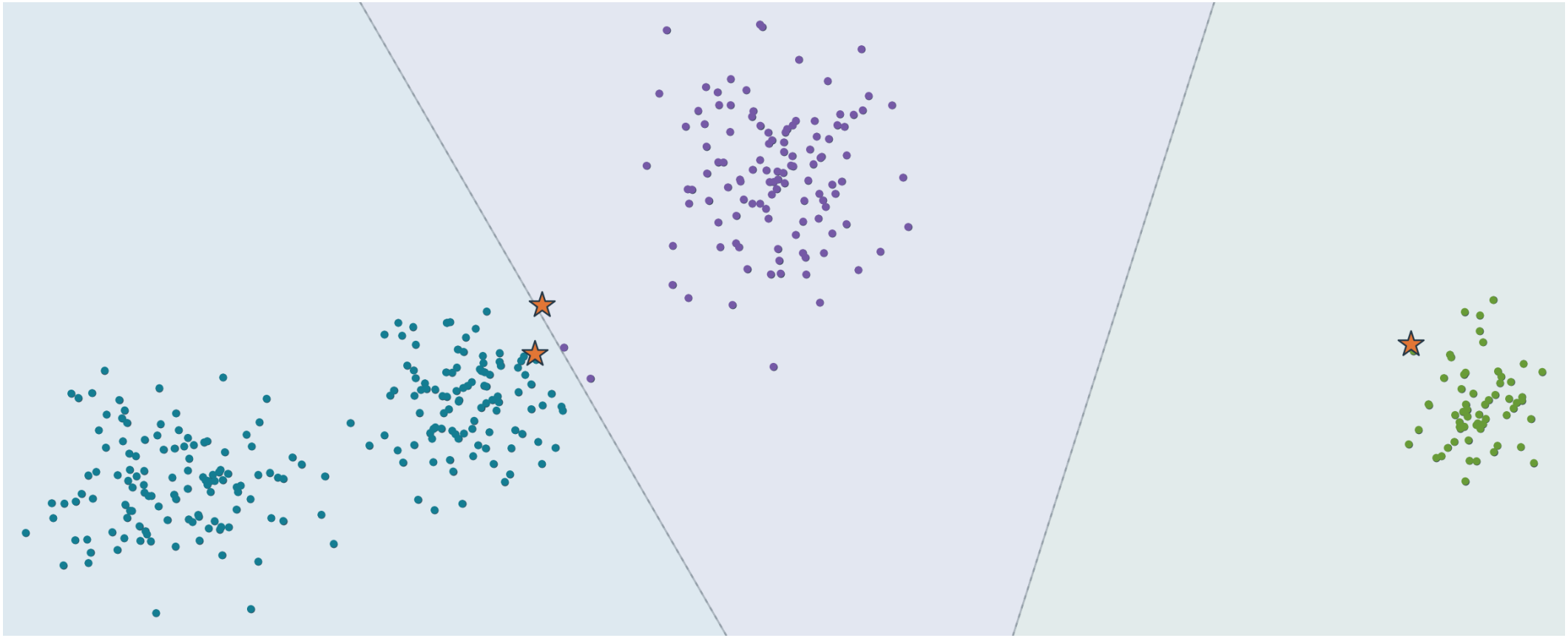
Clustering?

- Partition the data points into clusters, e.g., using k-means clustering
- Select one from each cluster…but how?



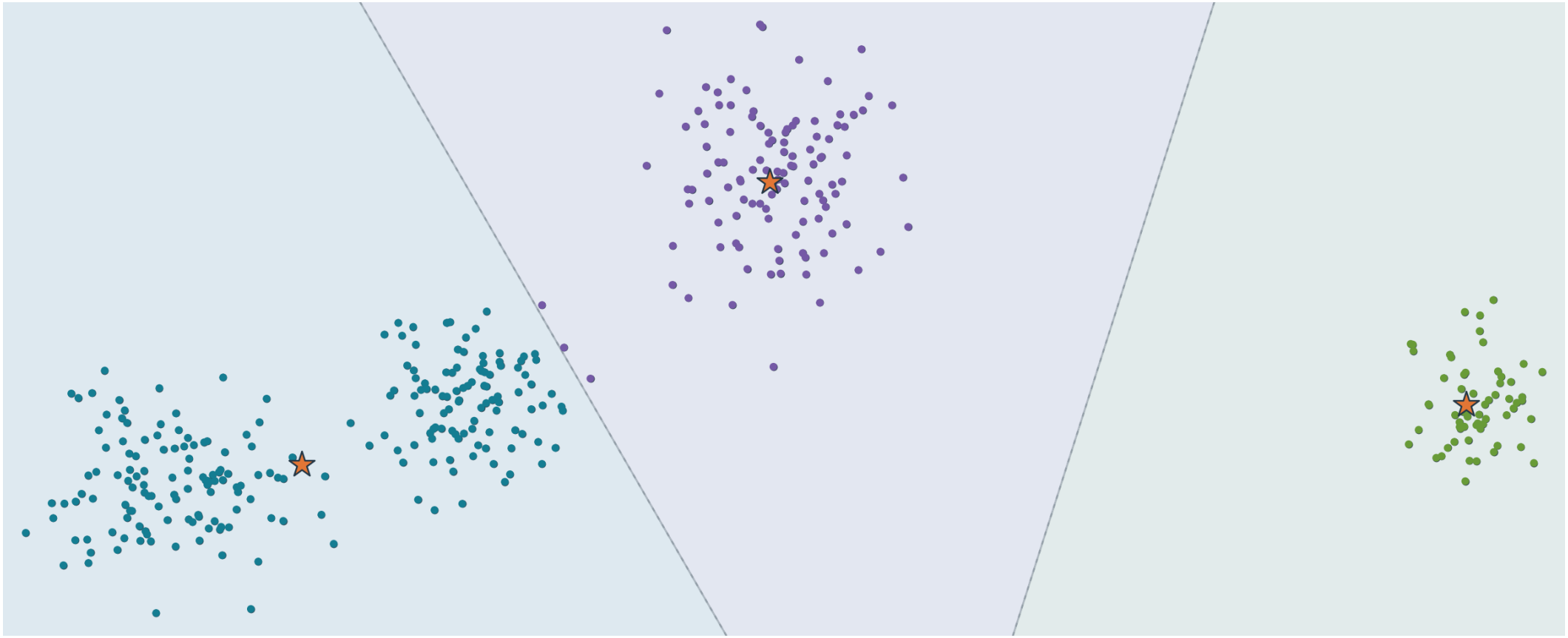
Clustering?

- Partition the data points into clusters, e.g., using k-means clustering
- Select one from each cluster…but how?



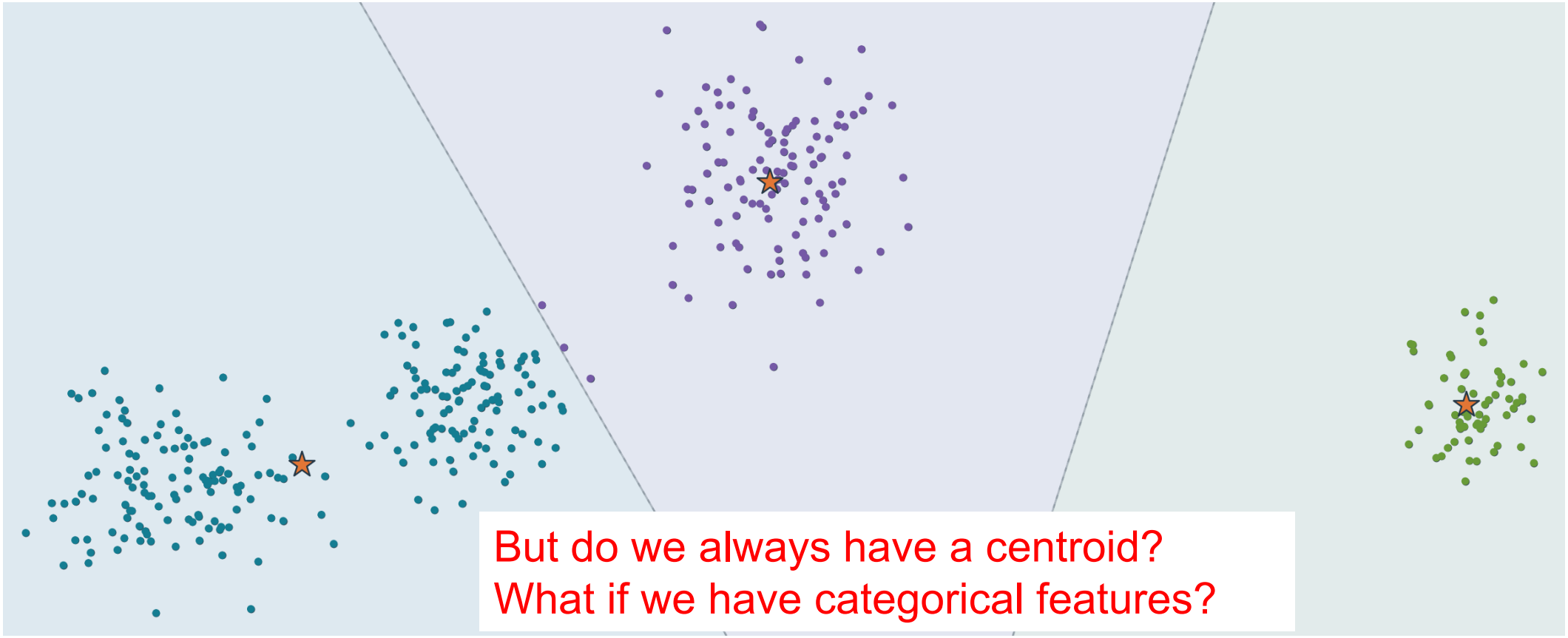
Clustering?

- Partition the data points into clusters, e.g., using k-means clustering
- Select one from each cluster…but how? One nearest to the centroid?



Clustering?

- Partition the data points into clusters, e.g., using k-means clustering
- Select one from each cluster…but how? One nearest to the centroid?



What Do We Want to Do in the First Place?

Define the problem before solving it!

What does “representative points” mean?

- Each of the data points is “close” to one of them?
- All data points is “close” to them on average?

How many points should we choose?

- Fixed number of points?
- As many as necessary?

Combinatorial Optimization

Find a combination such that

- **Constraints** are satisfied
- The **objective function** is optimized (minimized or maximized)

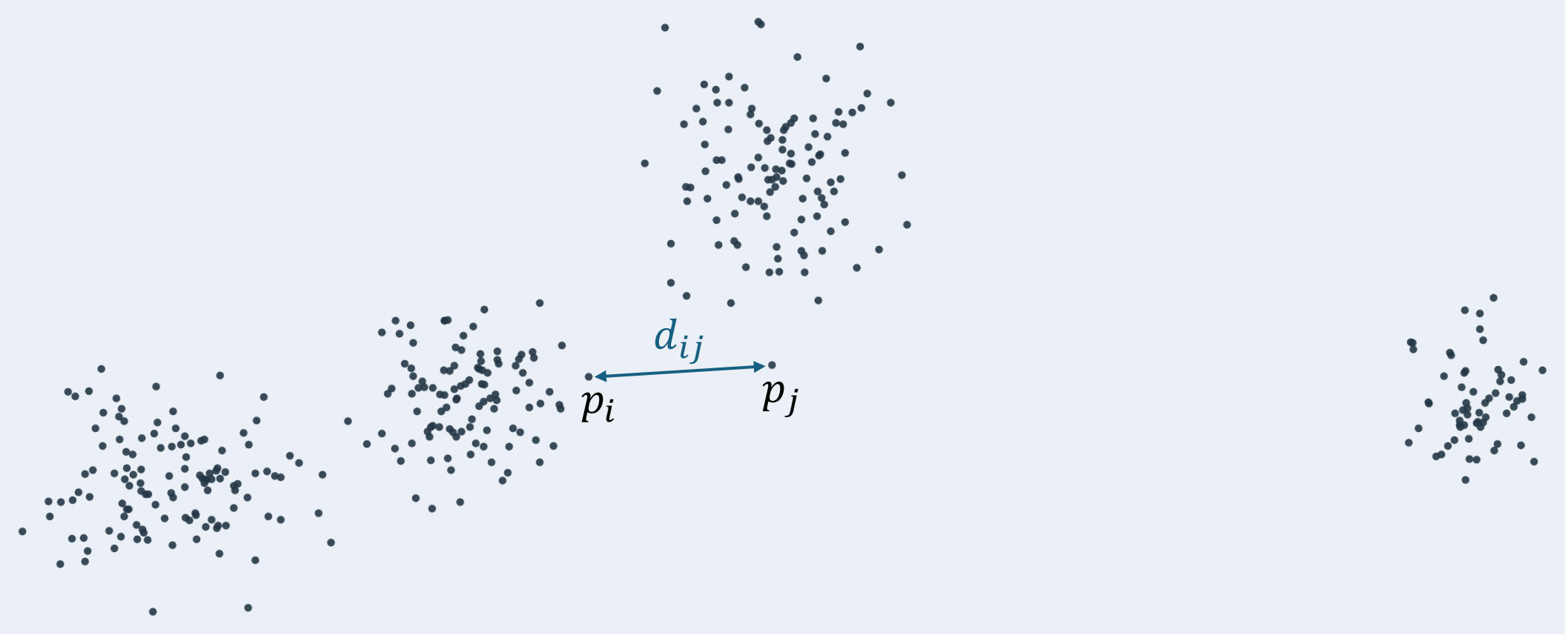
Formulate what we want to do as a combinatorial optimization problem and then solve it

We have multiple choices on

- Problem definition
- Parameter selection within the same problem definition

Mathematical Assumptions

- A set of n data points $P = \{p_1, \ldots, p_n\}$
- Distance d_{ij} between points p_i and p_j



Approach 1: Set Covering

Select a subset of data points $Q \subseteq P$ such that

- For each $p_j \in P$, there exists $p_i \in Q$ with $d_{ij} \leq r$
- The number of the selected data points $|Q|$ is minimized

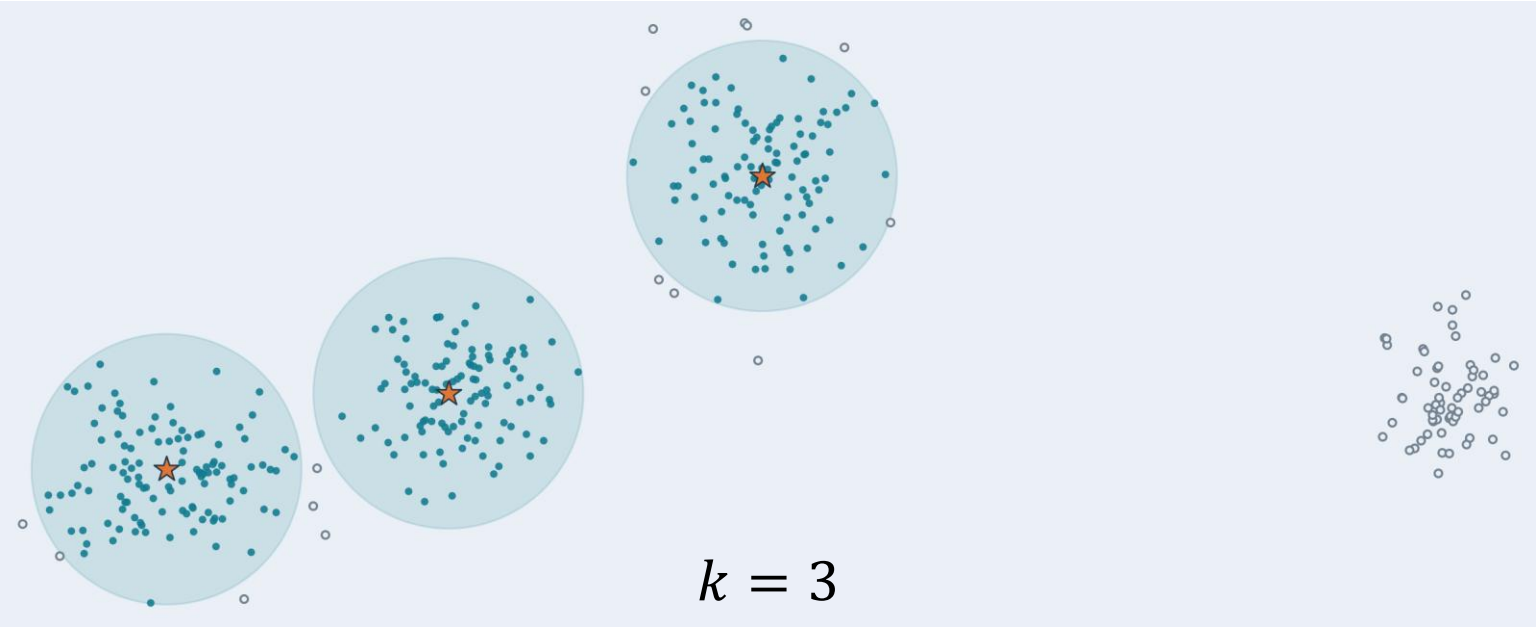
r is a parameter we decide



Approach 2: Maximum Coverage

Select a subset of data points $Q \subseteq P$ such that

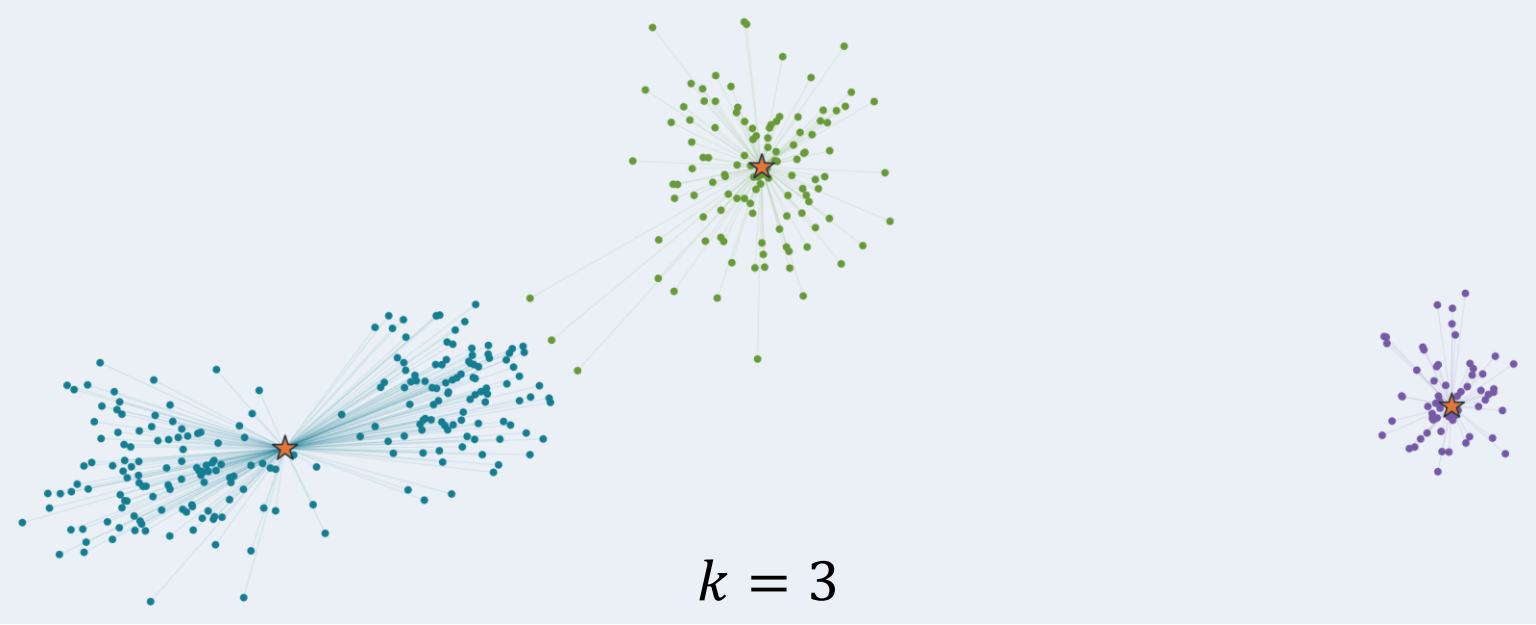
- Exactly k points are selected, i.e., $|Q| = k$
- The number of points covered with distance r, i.e., $\left|\{p_j \in P \mid \exists p_i \in Q, d_{ij} \leq r\}\right|$, is maximized



Approach 3: k-median (p-median)

Select a subset of data points $Q \subseteq P$ such that

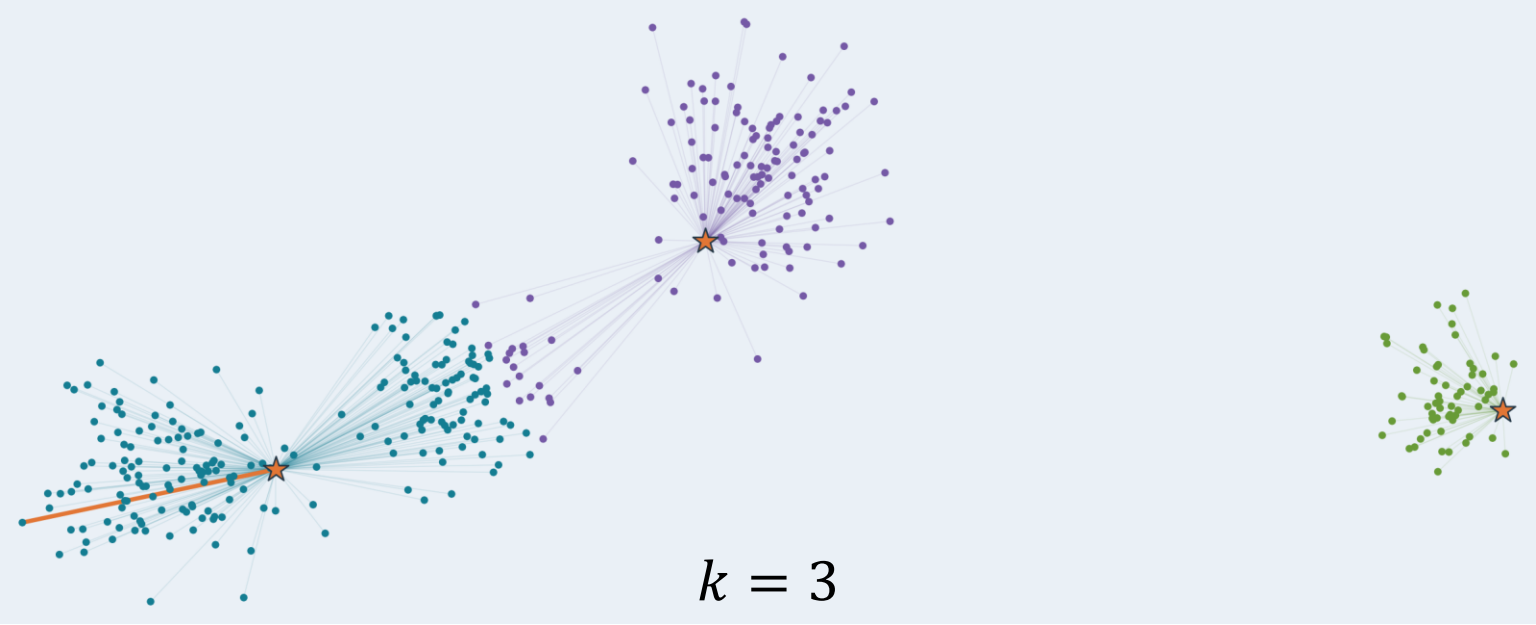
- Exactly k points are selected, i.e., $|Q| = k$
- The sum of distances to each point from the nearest selected point, i.e., $\sum_{p_j \in P} \min_{p_i \in Q} d_{ij}$, is minimized



Approach 4: k-center (p-center)

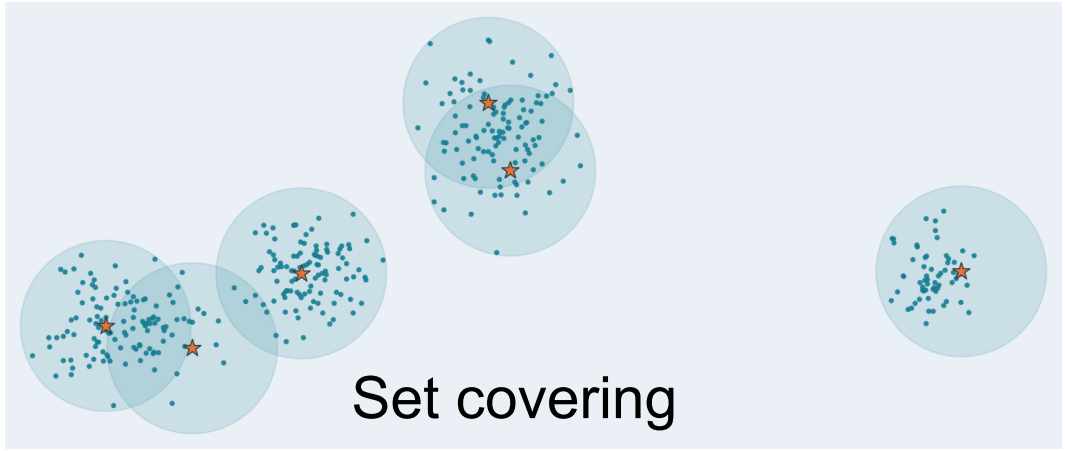
Select a subset of data points $Q \subseteq P$ such that

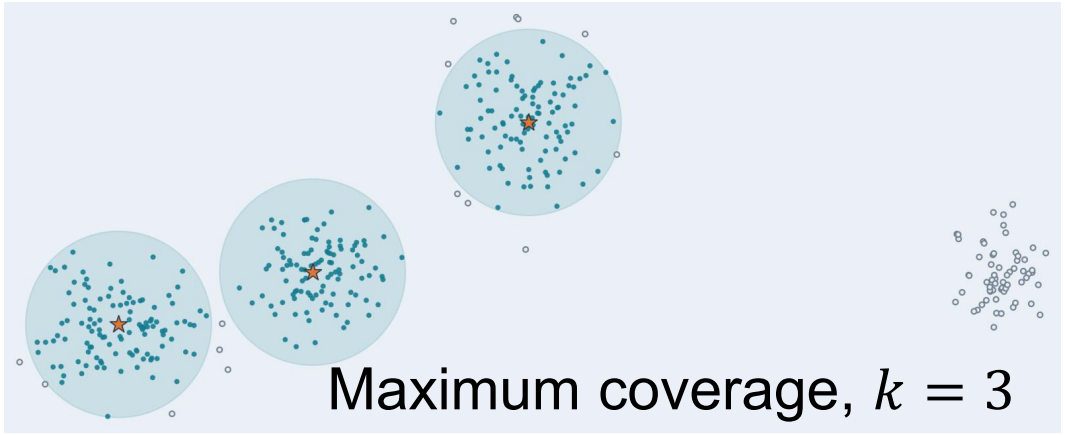
- Exactly k points are selected, i.e., $|Q| = k$
- The maximum distance to each point from the nearest selected point, i.e., $\max_{p_j \in P} \min_{p_i \in Q} d_{ij}$, is minimized

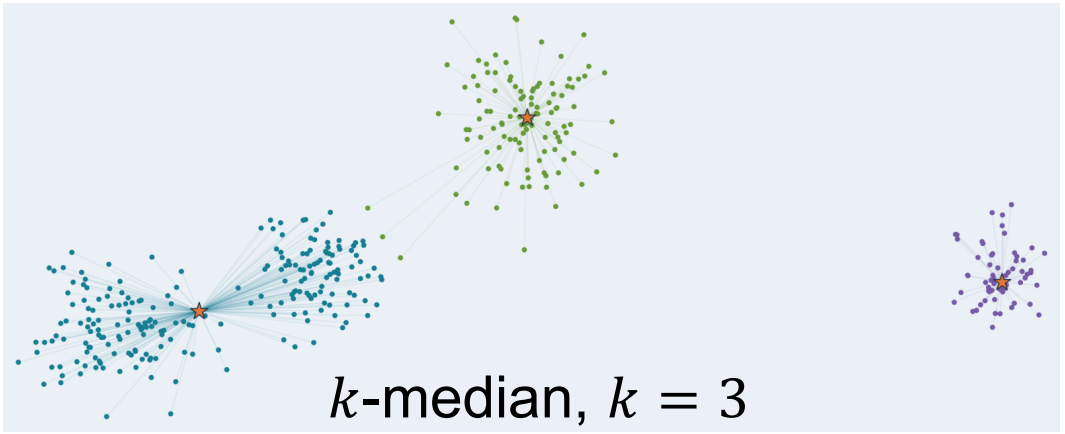


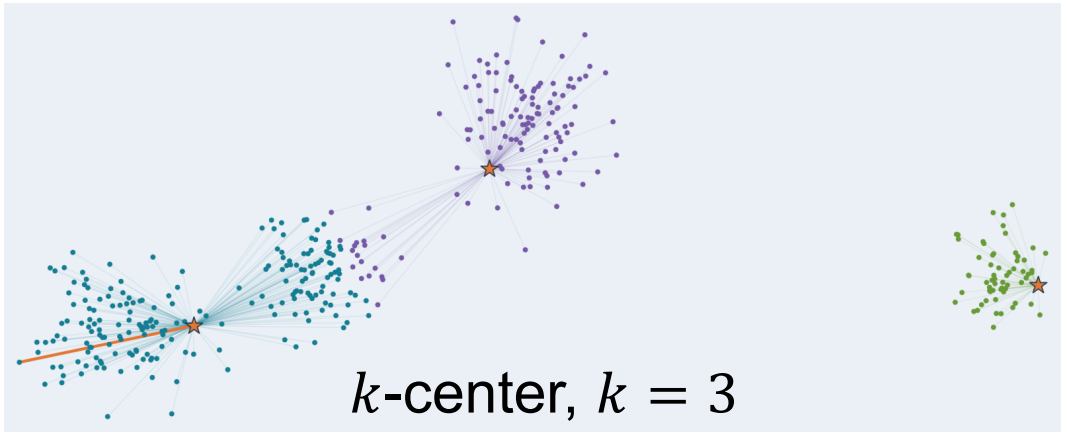
Problem Definition Matters

Different problem definitions result in different decisions









Solving

Exponential number of possible subsets (2^n or $\binom{n}{k}$)

Indeed, all the 4 problems are NP-hard even in the 2D Euclidean space!

References

- Karp, Reducibility among combinatorial problems, *Complexity of Computer Computations*, 85-103, 1972
- Papadimitrou, Worst-case and probabilistic analysis of a geometric location problem, *SIAM Journal on Computing* 10(3), 542-557, 1981
- Clark, Colbourn, and Johnson, Unit disk graphs, *Discrete Mathematics* 86, 165-177, 1990

Solving

Exponential number of possible subsets (2^n or $\binom{n}{k}$)

Indeed, all the 4 problems are NP-hard even in the 2D Euclidean space!

But this is not the end of the world!

- Practical algorithms, sometimes with an approximation guarantee
- Even free general-purpose solvers can work well in practice

Polynomial Time Approximation

Greedy algorithms with a known approximation guarantee

Problem	Reference	Time	Approximation ratio
Set covering	Chvatal 1979	$O(n^2)$	At most $\sum_{j=1}^{m} \frac{1}{j} \leq \log n$ times, where $m = \max_{p_i \in P} \lvert\{p_j \in P \mid d_{ij} \leq r\}\rvert$
Maximum coverage	Nemhauser et al., 1978	$O(n^2)$	At least $\left(1 - \left(1 - \frac{1}{k}\right)^k\right) \geq \left(1 - \frac{1}{e}\right)$ times
k-center	Gonzalez 1985	$O(kn)$	At most 2 times when the distances satisfy the triangle inequality, i.e., $d_{ij} \leq d_{ik} + d_{kj}$

Approximation algorithms for k-median also exist but are omitted for simplicity

Set Covering

Select a subset of data points $Q \subseteq P$ such that

- For each $p_j \in P$, there exists $p_i \in Q$ with $d_{ij} \leq r$
- The number of the selected data points $|Q|$ is minimized

r is a parameter we decide



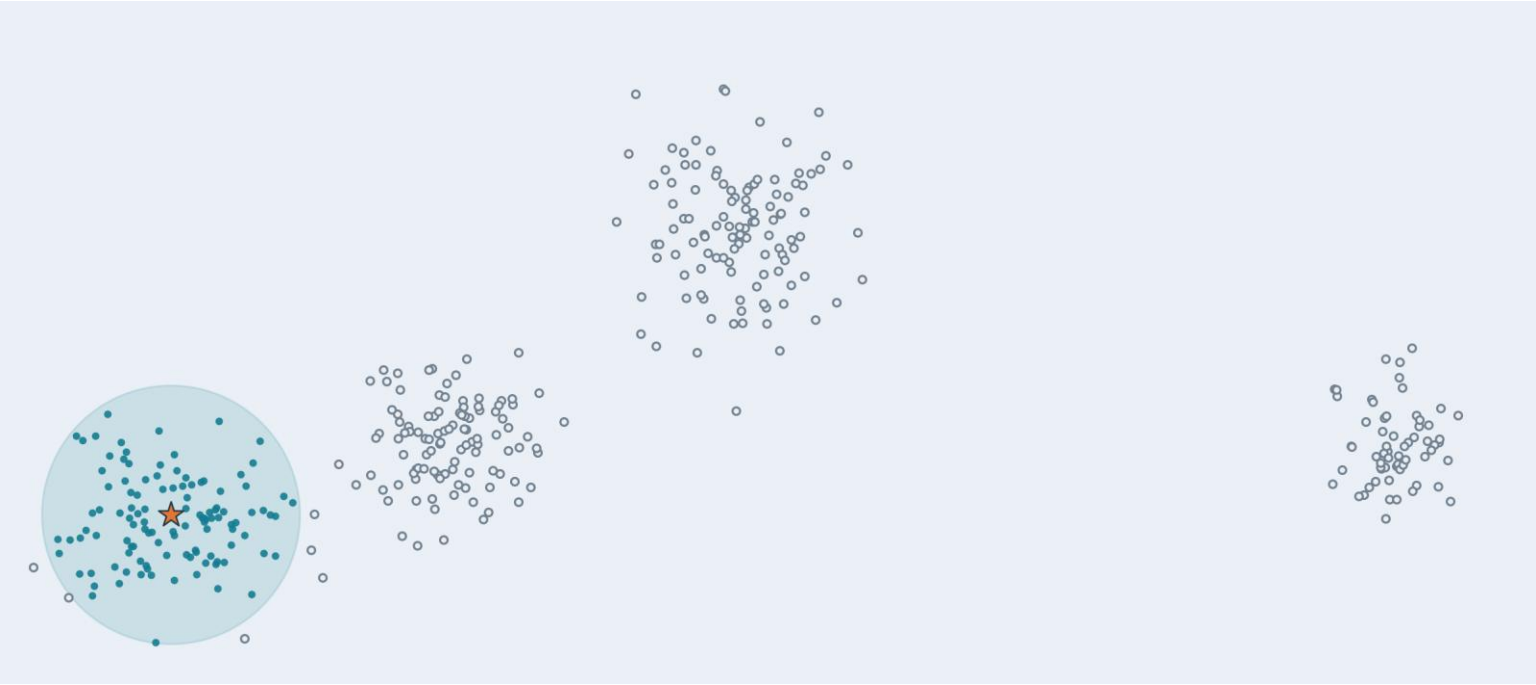
Greedy Algorithm for Set Covering

- Select a point that covers the largest number of uncovered points
- Repeat it until covering all points



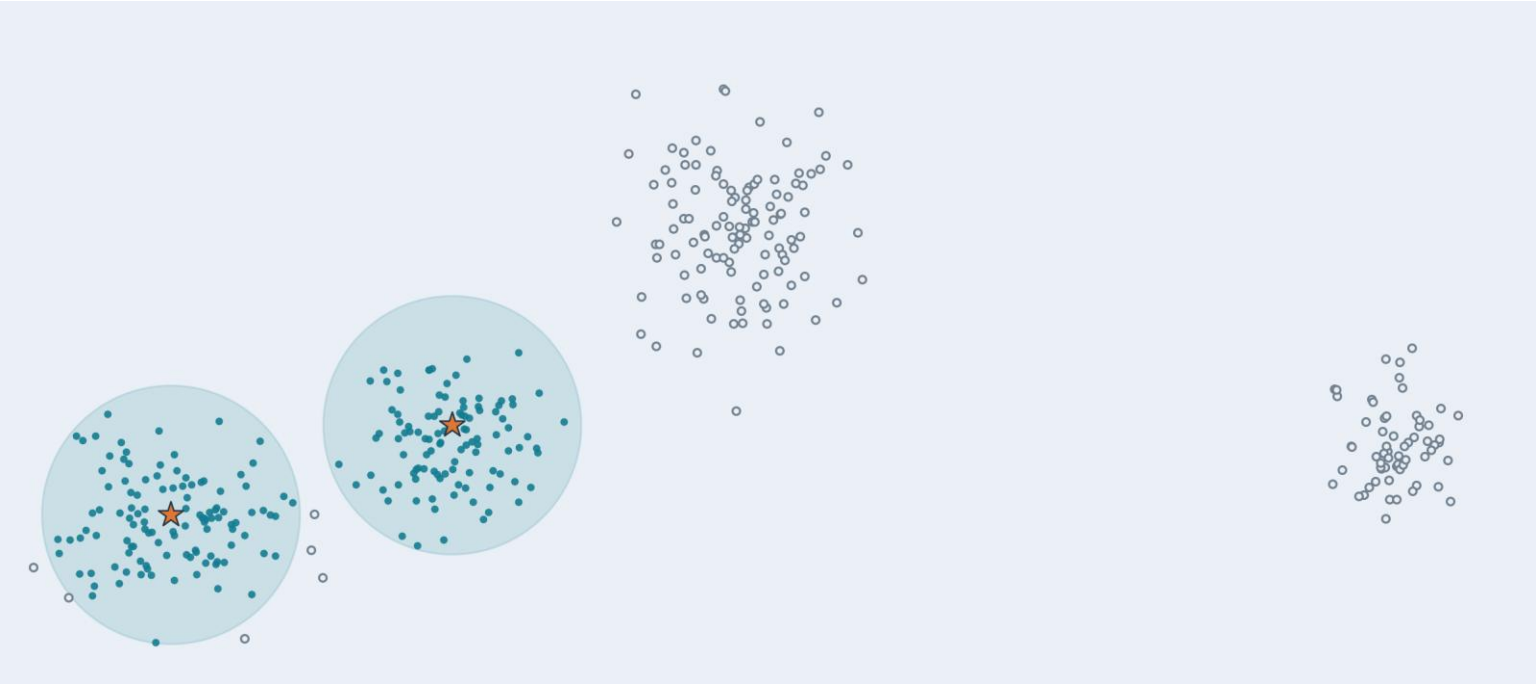
Greedy Algorithm for Set Covering

- Select a point that covers the largest number of uncovered points
- Repeat it until covering all points



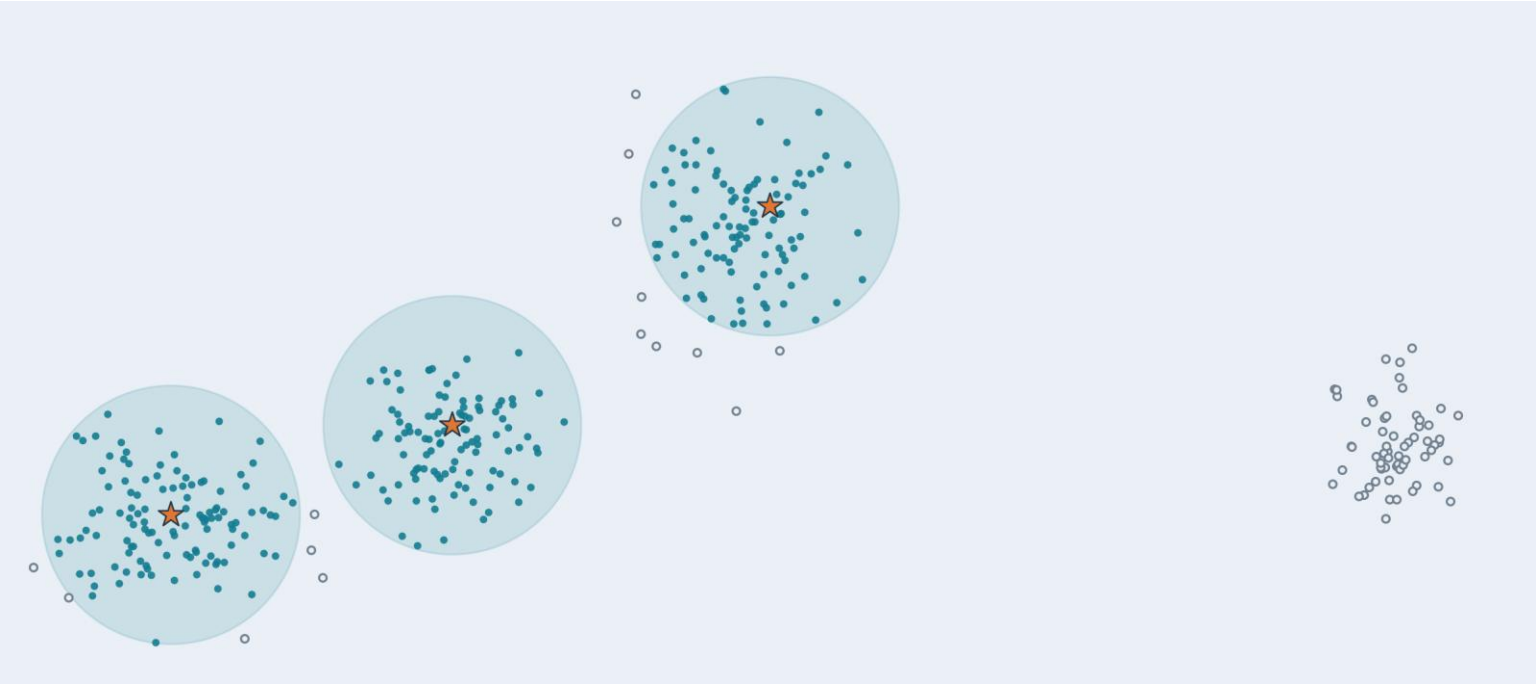
Greedy Algorithm for Set Covering

- Select a point that covers the largest number of uncovered points
- Repeat it until covering all points



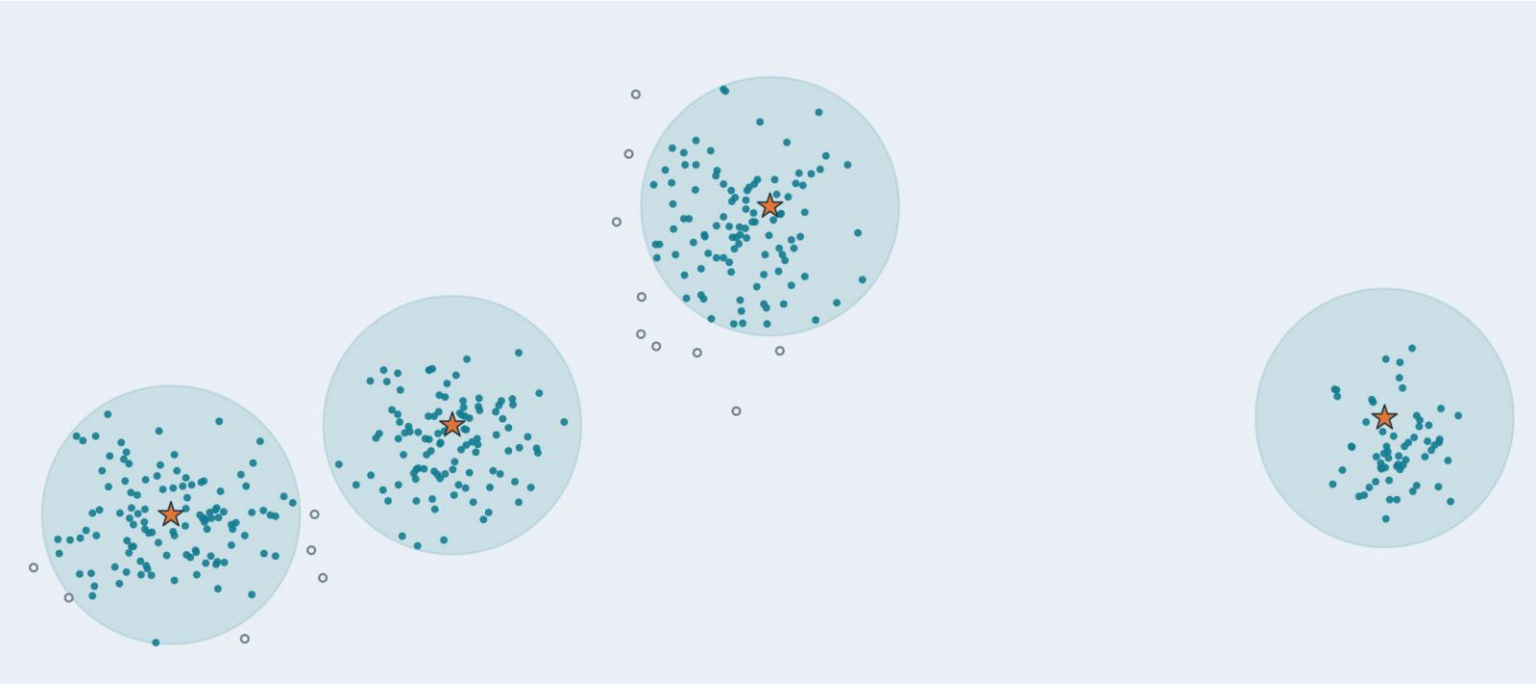
Greedy Algorithm for Set Covering

- Select a point that covers the largest number of uncovered points
- Repeat it until covering all points



Greedy Algorithm for Set Covering

- Select a point that covers the largest number of uncovered points
- Repeat it until covering all points



Greedy Algorithm for Set Covering

- Select a point that covers the largest number of uncovered points
- Repeat it until covering all points



Greedy Algorithm for Set Covering

- Select a point that covers the largest number of uncovered points
- Repeat it until covering all points



Greedy Algorithm for Set Covering

- Select a point that covers the largest number of uncovered points
- Repeat it until covering all points



Greedy Algorithm for Set Covering

- Select a point that covers the largest number of uncovered points
- Repeat it until covering all points

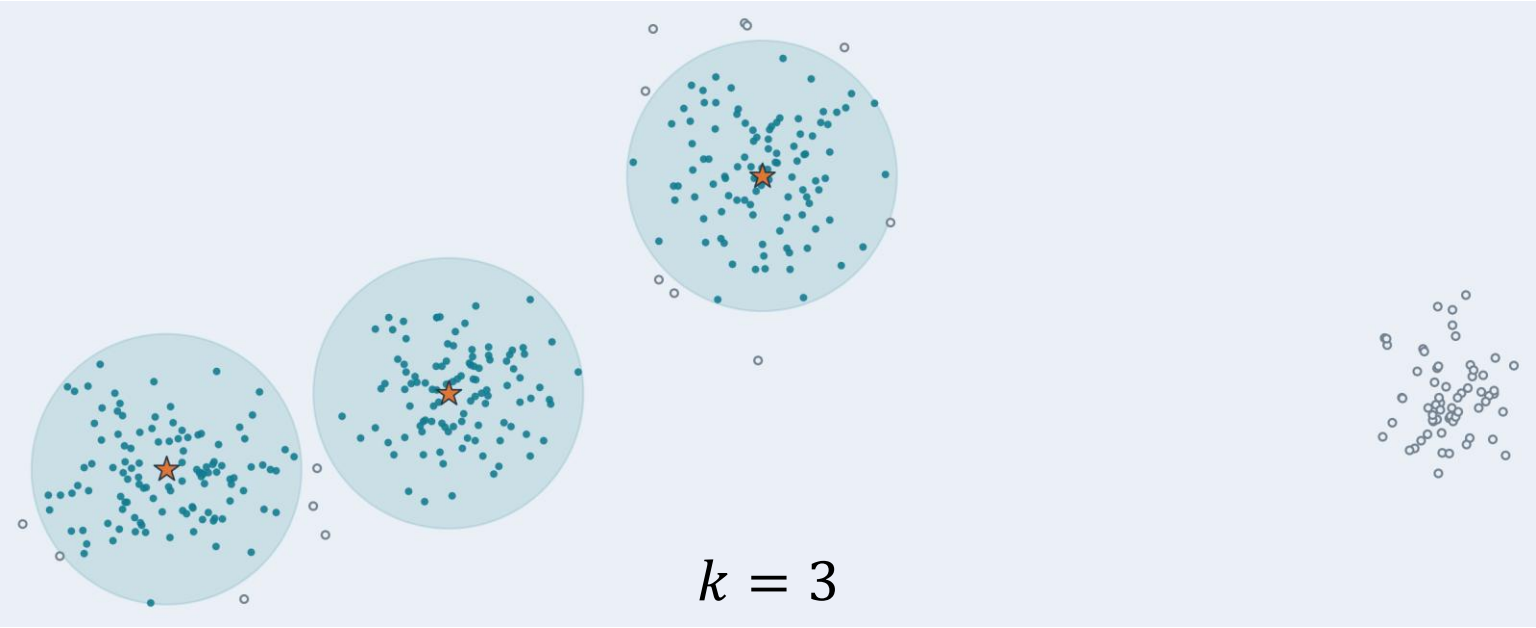
The number of selected points: 8



Maximum Coverage

Select a subset of data points $Q \subseteq P$ such that

- Exactly k points are selected, i.e., $|Q| = k$
- The number of points covered with distance r, i.e., $\left|\{p_j \in P \mid \exists p_i \in Q, d_{ij} \leq r\}\right|$, is maximized



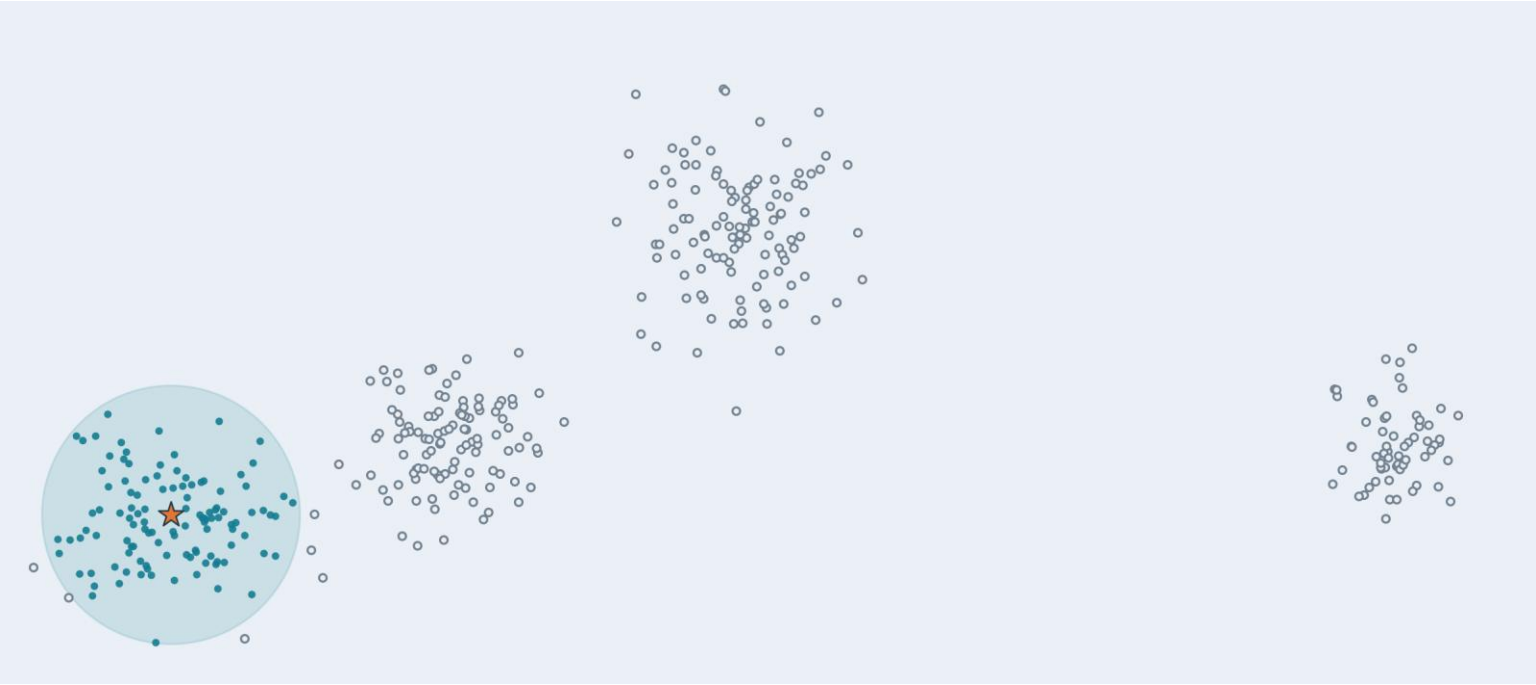
Greedy Algorithm for Maximum Coverage

- Select a point that covers the largest number of uncovered points
- Repeat it until selecting k points



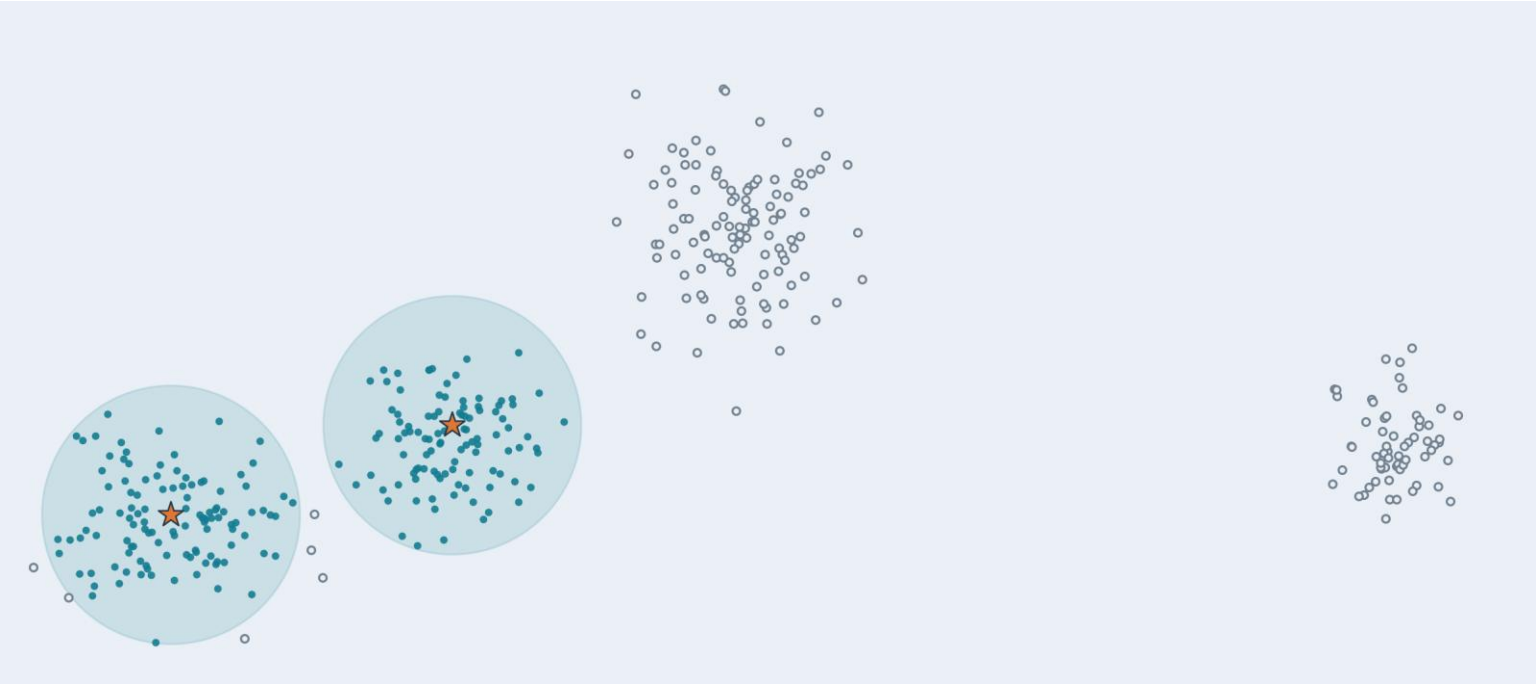
Greedy Algorithm for Maximum Coverage

- Select a point that covers the largest number of uncovered points
- Repeat it until selecting k points



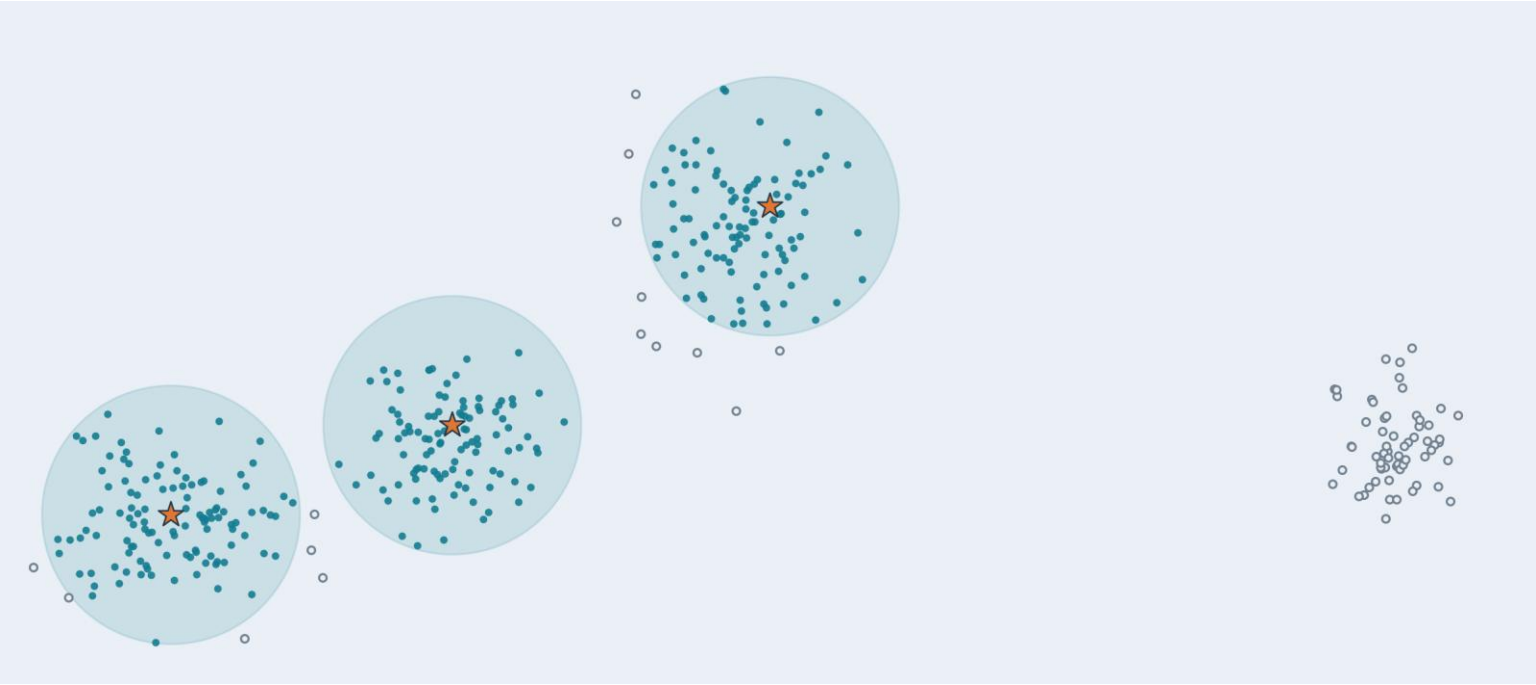
Greedy Algorithm for Maximum Coverage

- Select a point that covers the largest number of uncovered points
- Repeat it until selecting k points



Greedy Algorithm for Maximum Coverage

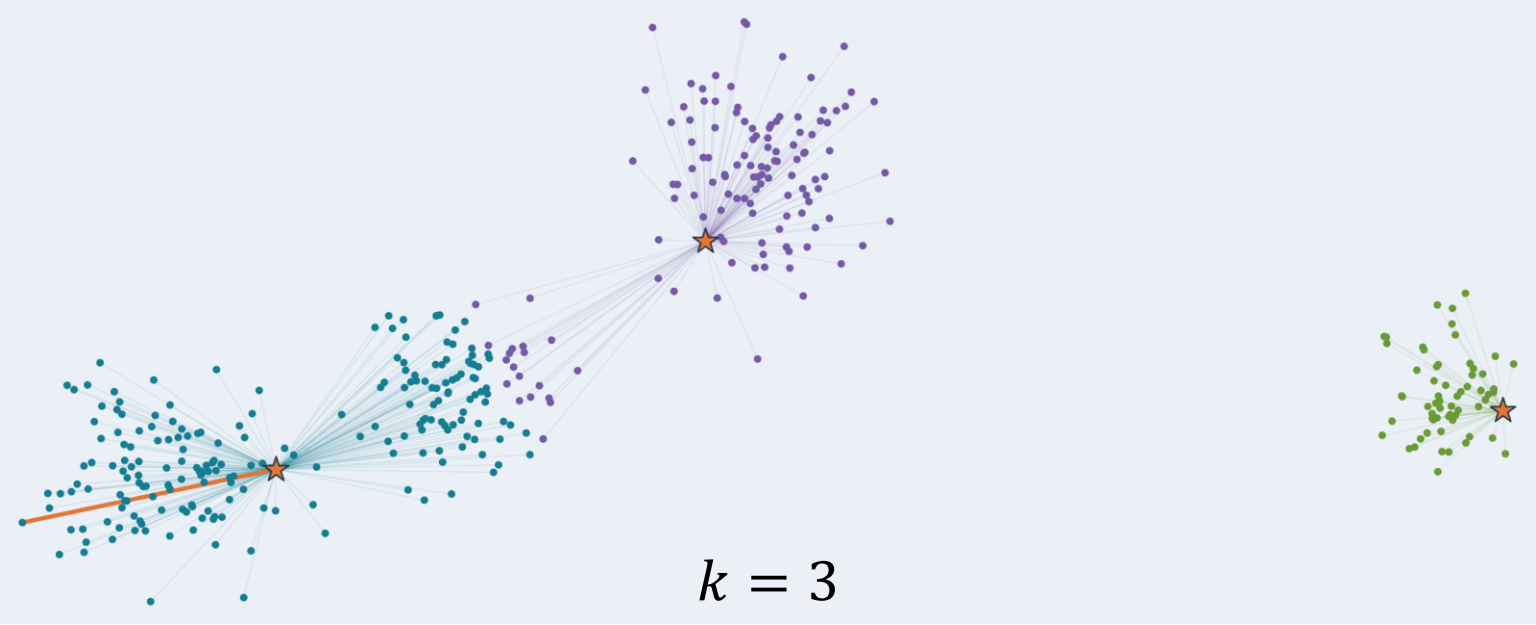
- Select a point that covers the largest number of uncovered points
- Repeat it until selecting k points



k-center

Select a subset of data points $Q \subseteq P$ such that

- Exactly k points are selected, i.e., $|Q| = k$
- The maximum distance to each point from the nearest selected point, i.e., $\max_{p_j \in P} \min_{p_i \in Q} d_{ij}$, is minimized



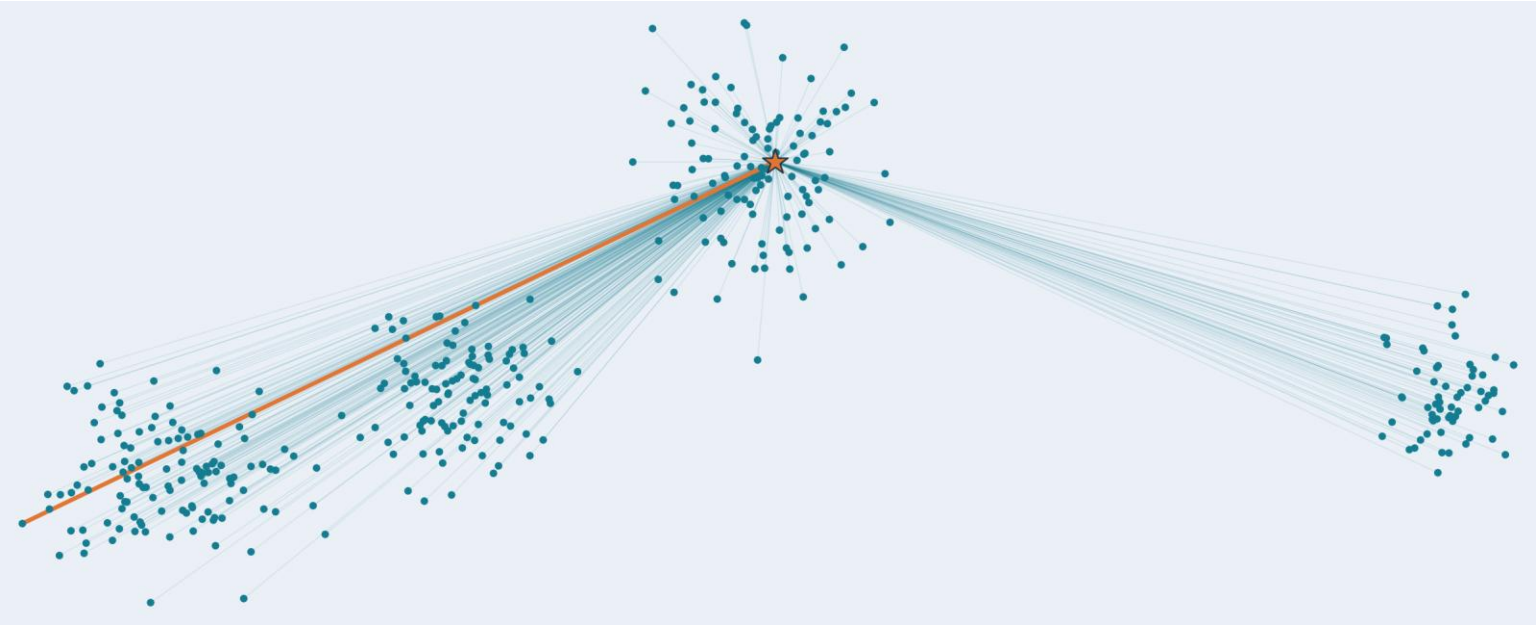
Greedy Algorithm for k-center

- Select the first point arbitrarily
- Select a point that is farthest from the selected points, i.e., point p_j maximizing $\min_{p_i \in Q} d_{ij}$ given the selected set Q, and repeat it $k-1$ times



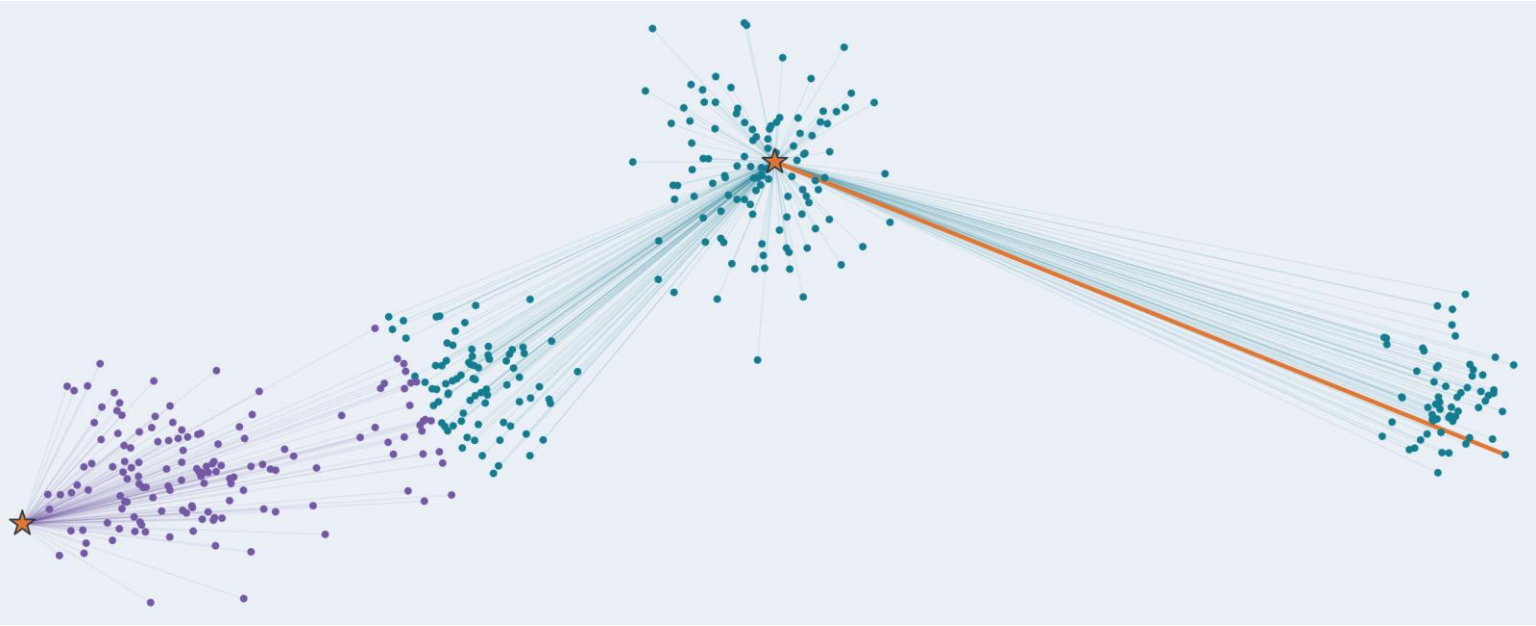
Greedy Algorithm for k-center

- Select the first point arbitrarily
- Select a point that is farthest from the selected points, i.e., point p_j maximizing $\min_{p_i \in Q} d_{ij}$ given the selected set Q, and repeat it $k-1$ times



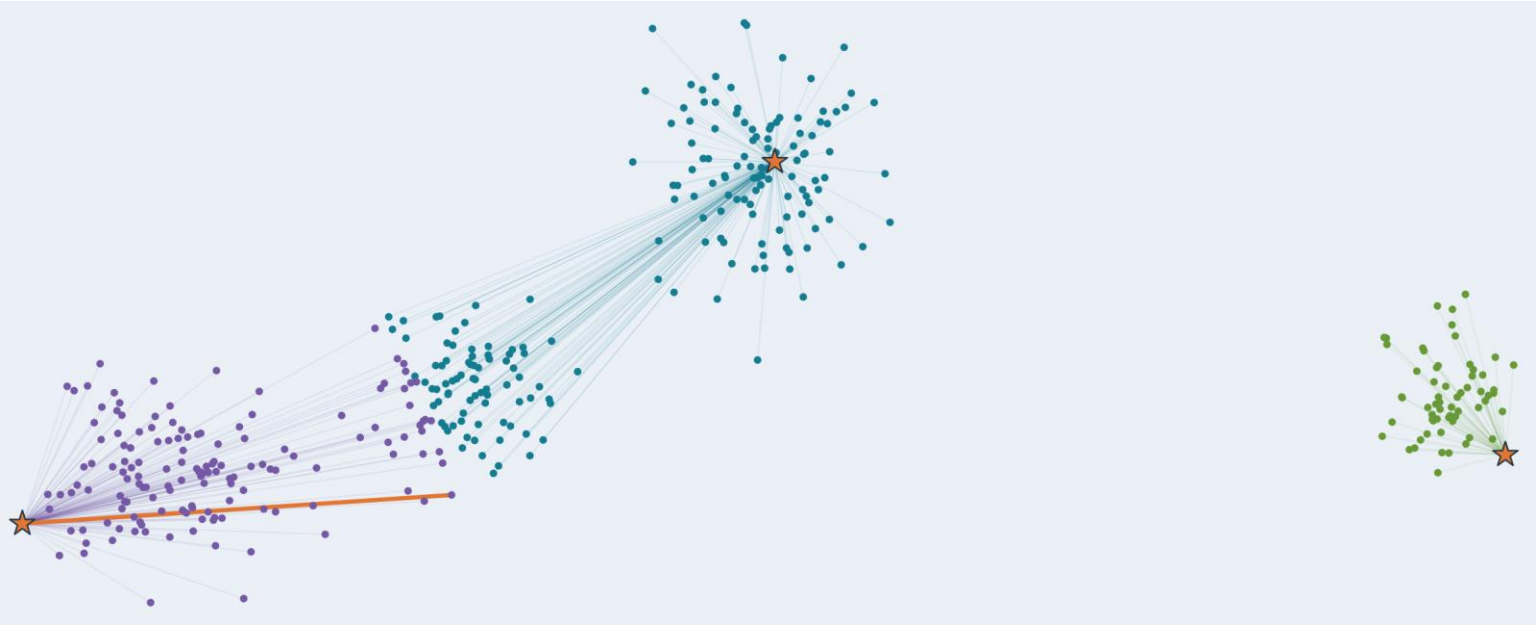
Greedy Algorithm for k-center

- Select the first point arbitrarily
- Select a point that is farthest from the selected points, i.e., point p_j maximizing $\min_{p_i \in Q} d_{ij}$ given the selected set Q, and repeat it $k-1$ times



Greedy Algorithm for k-center

- Select the first point arbitrarily
- Select a point that is farthest from the selected points, i.e., point p_j maximizing $\min_{p_i \in Q} d_{ij}$ given the selected set Q, and repeat it $k-1$ times

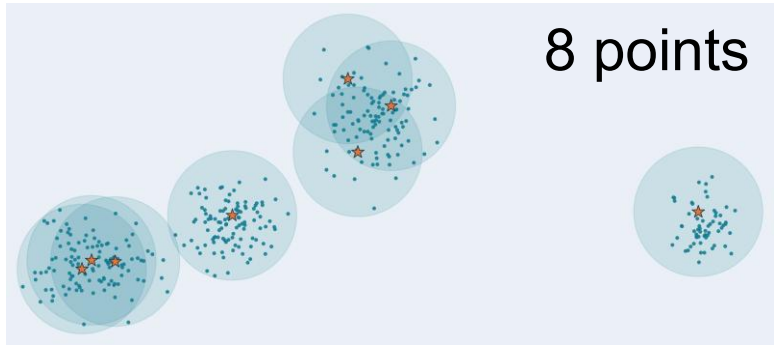


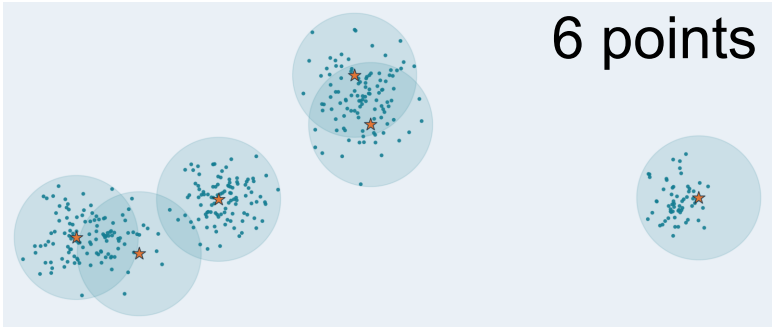
Greedy vs. Optimal Solutions

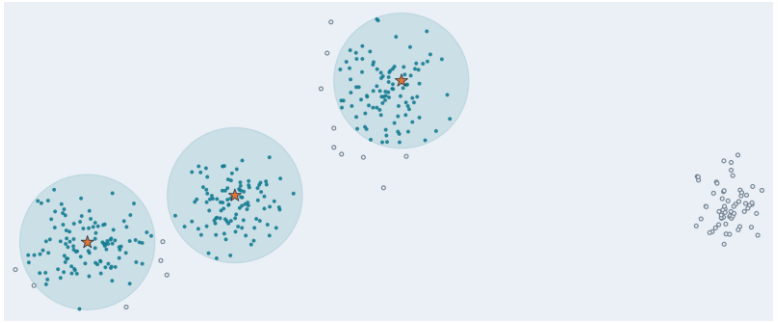
Set covering

Maximum coverage

k-center













Greedy

Optimal

Greedy vs. Optimal Solutions

Set covering

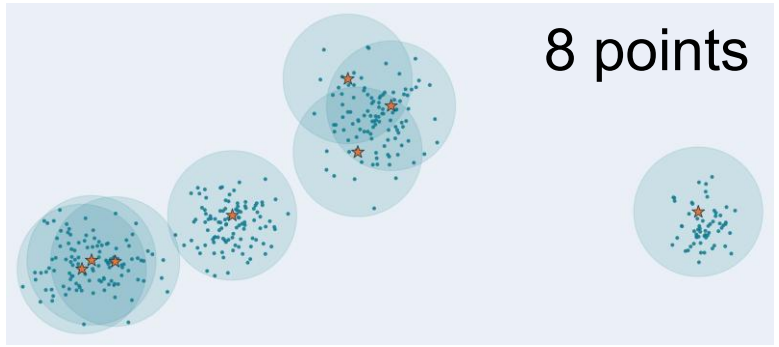
Terrible

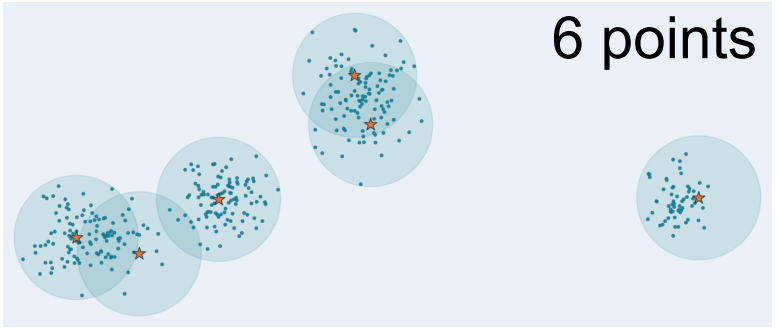
Maximum coverage

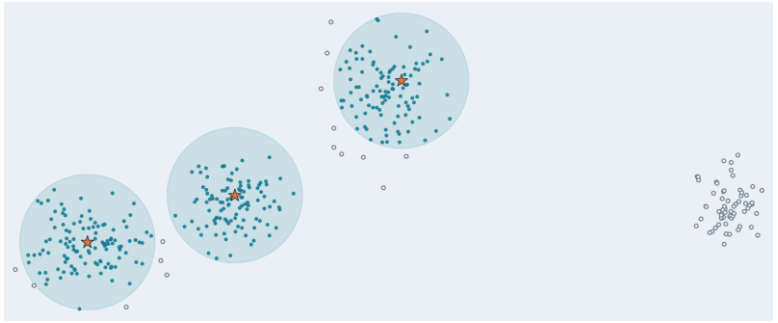
Almost the same

k-center

Not so bad













Greedy

Optimal

Greedy vs. Optimal Solutions

Set covering

Terrible





How to obtain an optimal solution?

Maximum coverage

Almost the same





k-center

Not so bad





Greedy

Optimal

Mixed-Integer Linear Programming (MILP)

Find the values of **decision variables** $x = (x_1, \ldots, x_n)$

- Minimizing the **linear objective function** $c^T x$ with $c \in \mathbb{R}^n$
- $x_1, \ldots, x_{n'} \in \mathbb{R}$ are **continuous variables**
- $x_{n'+1}, \ldots, x_n \in \mathbb{Z}$ are **integer variables**
- Satisfying **linear constraints** $Ax \leq b$ where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$

$$\begin{aligned} \min \ & c^T x \\ \text{s.t. } & Ax \leq b \\ & x \in \mathbb{R}^{n'} \times \mathbb{Z}^{n-n'} \end{aligned}$$

NP-hard in general, but open source and commercial general-purpose solvers are very effective in practice

Set Covering

Select a subset of data points $Q \subseteq P$ such that

- For each $p_j \in P$, there exists $p_i \in Q$ with $d_{ij} \leq r$
- The number of the selected data points $|Q|$ is minimized

r is a parameter we decide



MILP for Set Covering

- $a_{ij} = 1$ if $d_{ij} \leq r$ and 0 otherwise (a precomputed constant)
- $x_i = 1$ if point p_i is selected and 0 otherwise (a decision variable)

$$\min \sum_{i=1}^{n} x_i$$

The number of selected points

$$\text{s.t.} \sum_{i=1}^{n} a_{ij} x_i \geq 1, \quad j = 1, \ldots, n$$

Each point is covered

$$x_i \in \{0,1\}, \qquad i = 1, \ldots, n$$

Using a General-Purpose MILP Solver (SCIP)

```python
import pyscipopt as scip


def build_model(n, a, time_limit):
    model = scip.Model()

    x = [model.addVar(name=f"x_{i}", vtype="B", obj=1) for i in range(n)]
    model.addConss(
        scip.quicksum(a[i, j] * x[i] for i in range(n)) >= 1 for j in range(n)
    )
    model.setMinimize()

    model.setParam("limits/time", time_limit)
    model.optimize()
    solution = model.getBestSol()
    selected = (
        None
        if solution is None
        else [i for i in range(n) if model.getSolVal(solution, x[i]) > 0.5]
    )
    bound = model.getDualbound()

    return selected, bound
```

Using a General-Purpose MILP Solver (SCIP)

```python
import pyscipopt as scip
```

pip install pyscipopt
Open-source and free

```python
def build_model(n, a, time_limit):
    model = scip.Model()

    x = [model.addVar(name=f"x_{i}", vtype="B", obj=1) for i in range(n)]
    model.addConss(
        scip.quicksum(a[i, j] * x[i] for i in range(n)) >= 1 for j in range(n)
    )
    model.setMinimize()

    model.setParam("limits/time", time_limit)
    model.optimize()
    solution = model.getBestSol()
    selected = (
        None
        if solution is None
        else [i for i in range(n) if model.getSolVal(solution, x[i]) > 0.5]
    )
    bound = model.getDualbound()

    return selected, bound
```

Using a General-Purpose MILP Solver (SCIP)

```python
import pyscipopt as scip


def build_model(n, a, time_limit):
    model = scip.Model()
    x = [model.addVar(name=f"x_{i}", vtype="B", obj=1) for i in range(n)]
    model.addConss(
        scip.quicksum(a[i, j] * x[i] for i in range(n)) >= 1 for j in range(n)
    )
    model.setMinimize()

    model.setParam("limits/time", time_limit)
    model.optimize()
    solution = model.getBestSol()
    selected = (
        None
        if solution is None
        else [i for i in range(n) if model.getSolVal(solution, x[i]) > 0.5]
    )
    bound = model.getDualbound()

    return selected, bound
```

Create a model

Using a General-Purpose MILP Solver (SCIP)

```python
import pyscipopt as scip


def build_model(n, a, time_limit):
    model = scip.Model()

    x = [model.addVar(name=f"x_{i}", vtype="B", obj=1) for i in range(n)]
    model.addConss(
        scip.quicksum(a[i, j] * x[i] for i in range(n)) >= 1 for j in range(n)
    )
    model.setMinimize()

    model.setParam("limits/time", time_limit)
    model.optimize()
    solution = model.getBestSol()
    selected = (
        None
        if solution is None
        else [i for i in range(n) if model.getSolVal(solution, x[i]) > 0.5]
    )
    bound = model.getDualbound()

    return selected, bound
```

$x_i \in \{0,1\}, \quad i = 1, \ldots, n$

Using a General-Purpose MILP Solver (SCIP)

```python
import pyscipopt as scip


def build_model(n, a, time_limit):
    model = scip.Model()

    x = [model.addVar(name=f"x_{i}", vtype="B", obj=1) for i in range(n)]
    model.addConss(
        scip.quicksum(a[i, j] * x[i] for i in range(n)) >= 1 for j in range(n)
    )
    model.setMinimize()

    model.setParam("limits/time", time_limit)
    model.optimize()
    solution = model.getBestSol()
    selected = (
        None
        if solution is None
        else [i for i in range(n) if model.getSolVal(solution, x[i]) > 0.5]
    )
    bound = model.getDualbound()

    return selected, bound
```

$x_i \in \{0,1\}, \quad i = 1, \ldots, n$

Binary (0,1) variable

Using a General-Purpose MILP Solver (SCIP)

```python
import pyscipopt as scip


def build_model(n, a, time_limit):
    model = scip.Model()

    x = [model.addVar(name=f"x_{i}", vtype="B", obj=1) for i in range(n)]
    model.addConss(
        scip.quicksum(a[i, j] * x[i] for i in range(n)) >= 1 for j in range(n)
    )
    model.setMinimize()

    model.setParam("limits/time", time_limit)
    model.optimize()
    solution = model.getBestSol()
    selected = (
        None
        if solution is None
        else [i for i in range(n) if model.getSolVal(solution, x[i]) > 0.5]
    )
    bound = model.getDualbound()

    return selected, bound
```

$\min \sum_{i=1}^{n} x_i$

Using a General-Purpose MILP Solver (SCIP)

```python
import pyscipopt as scip


def build_model(n, a, time_limit):
    model = scip.Model()

    x = [model.addVar(name=f"x_{i}", vtype="B", obj=1) for i in range(n)]
    model.addConss(
        scip.quicksum(a[i, j] * x[i] for i in range(n)) >= 1 for j in range(n)
    )
    model.setMinimize()

    model.setParam("limits/time", time_limit)
    model.optimize()
    solution = model.getBestSol()
    selected = (
        None
        if solution is None
        else [i for i in range(n) if model.getSolVal(solution, x[i]) > 0.5]
    )
    bound = model.getDualbound()

    return selected, bound
```

$$\sum_{i=1}^{n} a_{ij}\, x_i \geq 1,\ j = 1, \ldots, n$$

Using a General-Purpose MILP Solver (SCIP)

```python
import pyscipopt as scip


def build_model(n, a, time_limit):
    model = scip.Model()

    x = [model.addVar(name=f"x_{i}", vtype="B", obj=1) for i in range(n)]
    model.addConss(
        scip.quicksum(a[i, j] * x[i] for i in range(n)) >= 1 for j in range(n)
    )
    model.setMinimize()

    model.setParam("limits/time", time_limit)
    model.optimize()
    solution = model.getBestSol()
    selected = (
        None
        if solution is None
        else [i for i in range(n) if model.getSolVal(solution, x[i]) > 0.5]
    )
    bound = model.getDualbound()

    return selected, bound
```

Set time limit

Using a General-Purpose MILP Solver (SCIP)

```python
import pyscipopt as scip


def build_model(n, a, time_limit):
    model = scip.Model()

    x = [model.addVar(name=f"x_{i}", vtype="B", obj=1) for i in range(n)]
    model.addConss(
        scip.quicksum(a[i, j] * x[i] for i in range(n)) >= 1 for j in range(n)
    )
    model.setMinimize()

    model.setParam("limits/time", time_limit)
    model.optimize()
    solution = model.getBestSol()
    selected = (
        None
        if solution is None
        else [i for i in range(n) if model.getSolVal(solution, x[i]) > 0.5]
    )
    bound = model.getDualbound()

    return selected, bound
```

Run the solver

Using a General-Purpose MILP Solver (SCIP)

```python
import pyscipopt as scip


def build_model(n, a, time_limit):
    model = scip.Model()

    x = [model.addVar(name=f"x_{i}", vtype="B", obj=1) for i in range(n)]
    model.addConss(
        scip.quicksum(a[i, j] * x[i] for i in range(n)) >= 1 for j in range(n)
    )
    model.setMinimize()

    model.setParam("limits/time", time_limit)
    model.optimize()
    solution = model.getBestSol()
    selected = (
        None
        if solution is None
        else [i for i in range(n) if model.getSolVal(solution, x[i]) > 0.5]
    )
    bound = model.getDualbound()

    return selected, bound
```

Get the best solution found

Using a General-Purpose MILP Solver (SCIP)

```python
import pyscipopt as scip


def build_model(n, a, time_limit):
    model = scip.Model()

    x = [model.addVar(name=f"x_{i}", vtype="B", obj=1) for i in range(n)]
    model.addConss(
        scip.quicksum(a[i, j] * x[i] for i in range(n)) >= 1 for j in range(n)
    )
    model.setMinimize()

    model.setParam("limits/time", time_limit)
    model.optimize()
    solution = model.getBestSol()
    selected = (
        None
        if solution is None
        else [i for i in range(n) if model.getSolVal(solution, x[i]) > 0.5]
    )
    bound = model.getDualbound()

    return selected, bound
```

Extract the selected points

Using a General-Purpose MILP Solver (SCIP)

```python
import pyscipopt as scip


def build_model(n, a, time_limit):
    model = scip.Model()

    x = [model.addVar(name=f"x_{i}", vtype="B", obj=1) for i in range(n)]
    model.addConss(
        scip.quicksum(a[i, j] * x[i] for i in range(n)) >= 1 for j in range(n)
    )
    model.setMinimize()

    model.setParam("limits/time", time_limit)
    model.optimize()
    solution = model.getBestSol()
    selected = (
        None
        if solution is None
        else [i for i in range(n) if model.getSolVal(solution, x[i]) > 0.5]
    )
    bound = model.getDualbound()

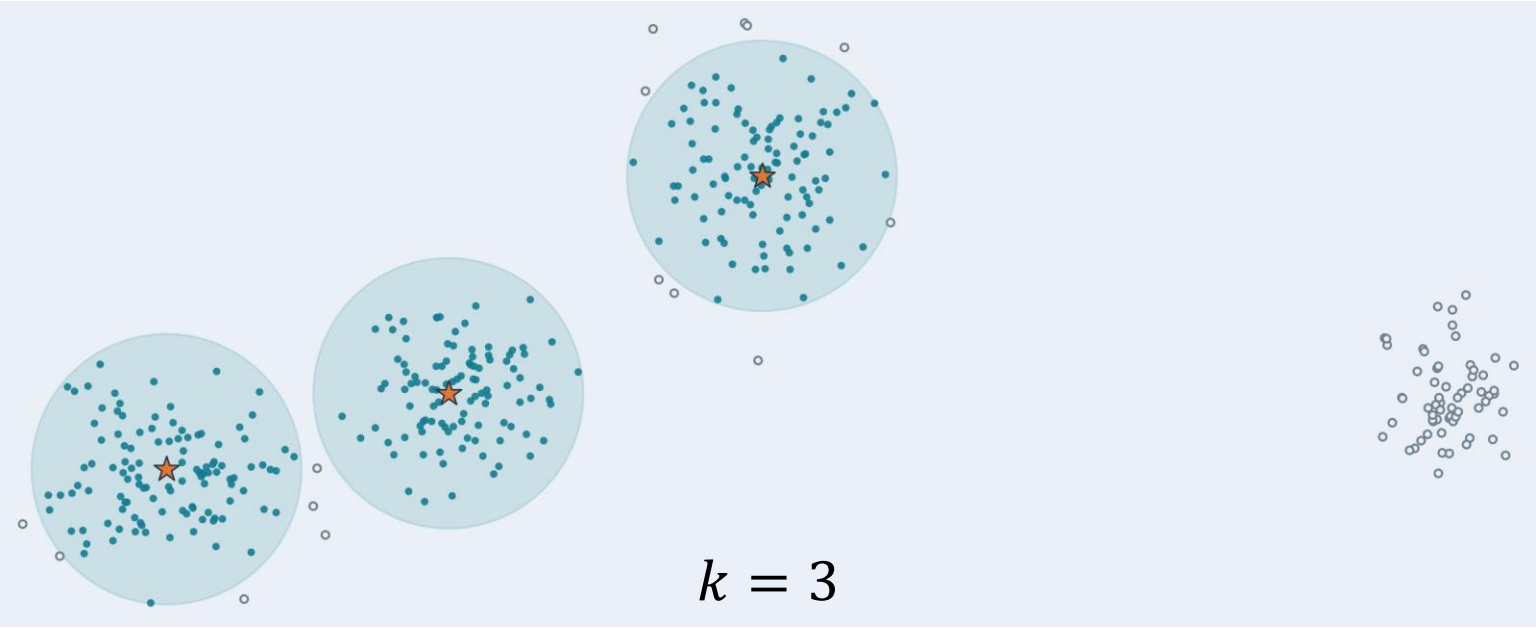
    return selected, bound
```

Get a lower bound on the number of points

Maximum Coverage

Select a subset of data points $Q \subseteq P$ such that

- Exactly k points are selected, i.e., $|Q| = k$
- The number of points covered with distance r, i.e., $\left|\{p_j \in P \mid \exists p_i \in Q, d_{ij} \leq r\}\right|$, is maximized



MILP for Maximum Coverage

- $a_{ij} = 1$ if $d_{ij} \leq r$ and 0 otherwise (a precomputed constant)
- $x_i = 1$ if point p_i is selected and 0 otherwise (a decision variable)
- $y_i = 1$ if point p_i is covered and 0 otherwise (a decision variable)

$$\max \sum_{i=1}^{n} y_i$$

The number of covered points

$$\text{s.t.} \sum_{i=1}^{n} x_i \leq k$$

At most k points are selected

$$\sum_{i=1}^{n} a_{ij} x_i \geq y_j, \quad j = 1, \ldots, n$$

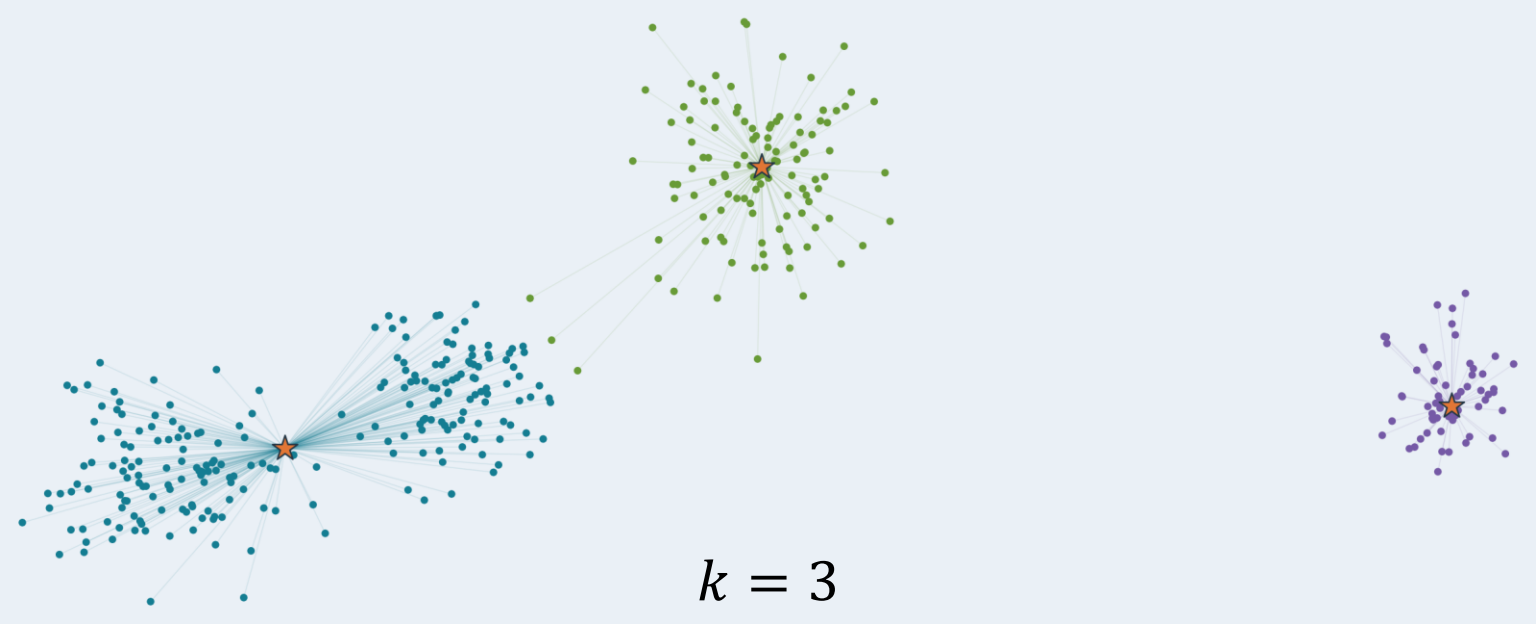
Link selected and covered points

$$x_i, y_i \in \{0,1\}, \quad i = 1, \ldots, n$$

k-median

Select a subset of data points $Q \subseteq P$ such that

- Exactly k points are selected, i.e., $|Q| = k$
- The sum of distances to each point from the nearest selected point, i.e., $\sum_{p_j \in P} \min_{p_i \in Q} d_{ij}$, is minimized

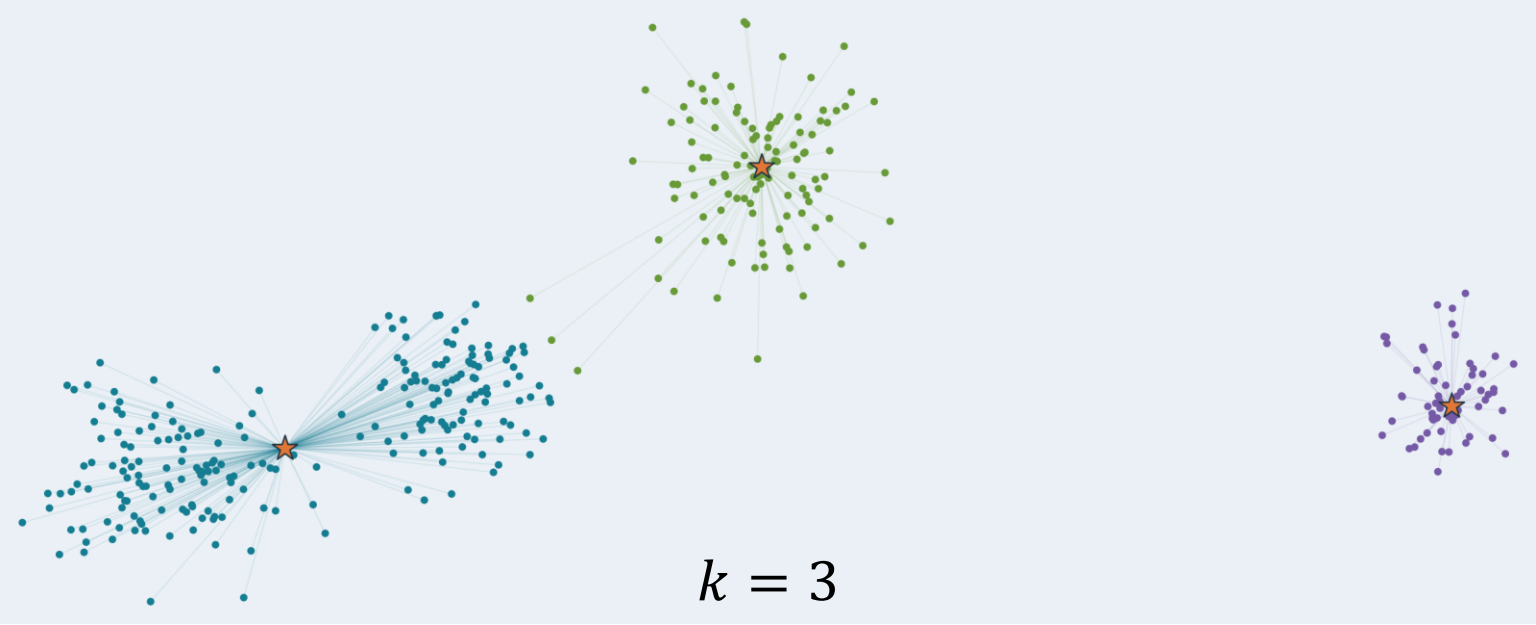


k-median

Select a subset of data points $Q \subseteq P$ such that

- Exactly k points are selected, i.e., $|Q| = k$
- The sum of distances to each point from the nearest selected point, i.e., $\sum_{p_j \in P} \min_{p_i \in Q} d_{ij}$, is minimized

Nonlinear objective!



MILP for k-median

- $x_i = 1$ if point p_i is selected and 0 otherwise (a decision variable)

$$\min$$

$$\text{s. t.} \sum_{i=1}^{n} x_i \leq k$$

At most k points are selected

$$x_i, \in \{0,1\}, \quad i = 1, \ldots, n$$

MILP for k-median

- $x_i = 1$ if point p_i is selected and 0 otherwise (a decision variable)
- $z_{ij} = 1$ if p_j is assigned to p_i and 0 otherwise (a decision variable)

$$\min \sum_{i=1}^{n} \sum_{j=1}^{n} d_{ij} z_{ij}$$

Sum of the distances to the assigned point

$$\text{s.t.} \sum_{i=1}^{n} x_i \leq k$$

At most k points are selected

$$\sum_{i=1}^{n} z_{ij} = 1, \quad j = 1, \ldots, n$$

Assigned to one selected point

$$z_{ij} \leq x_i, \quad i, j = 1, \ldots, n$$

Only assigned to a selected point

$$x_i, \in \{0,1\}, \quad i = 1, \ldots, n$$

$$z_{ij}, \in \{0,1\}, \quad i, j = 1, \ldots, n$$

MILP for k-median

- $x_i = 1$ if point p_i is selected and 0 otherwise (a decision variable)
- $z_{ij} = 1$ if p_j is assigned to p_i and 0 otherwise (a decision variable)

Minimization ensures that each point is assigned to the nearest one

$$\min \sum_{i=1}^{n} \sum_{j=1}^{n} d_{ij} z_{ij}$$

Sum of the distances to the assigned point

$$\text{s.t.} \sum_{i=1}^{n} x_i \leq k$$

At most k points are selected

$$\sum_{i=1}^{n} z_{ij} = 1, \quad j = 1, \ldots, n$$

Assigned to one selected point

$$z_{ij} \leq x_i, \quad i, j = 1, \ldots, n$$

Only assigned to a selected point

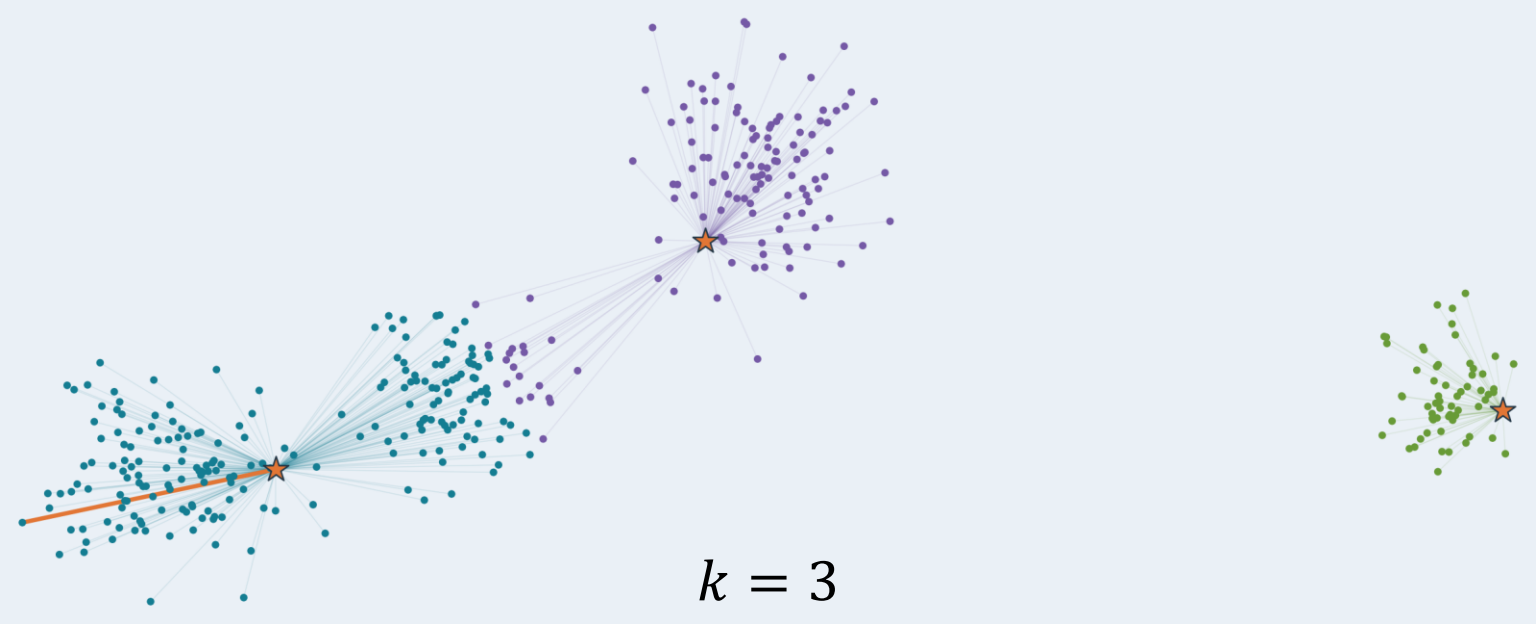
$$x_i, \in \{0,1\}, \quad i = 1, \ldots, n$$

$$z_{ij}, \in \{0,1\}, \quad i, j = 1, \ldots, n$$

k-center

Select a subset of data points $Q \subseteq P$ such that

- Exactly k points are selected, i.e., $|Q| = k$
- The maximum distance to each point from the nearest selected point, i.e., $\max_{p_j \in P} \min_{p_i \in Q} d_{ij}$, is minimized

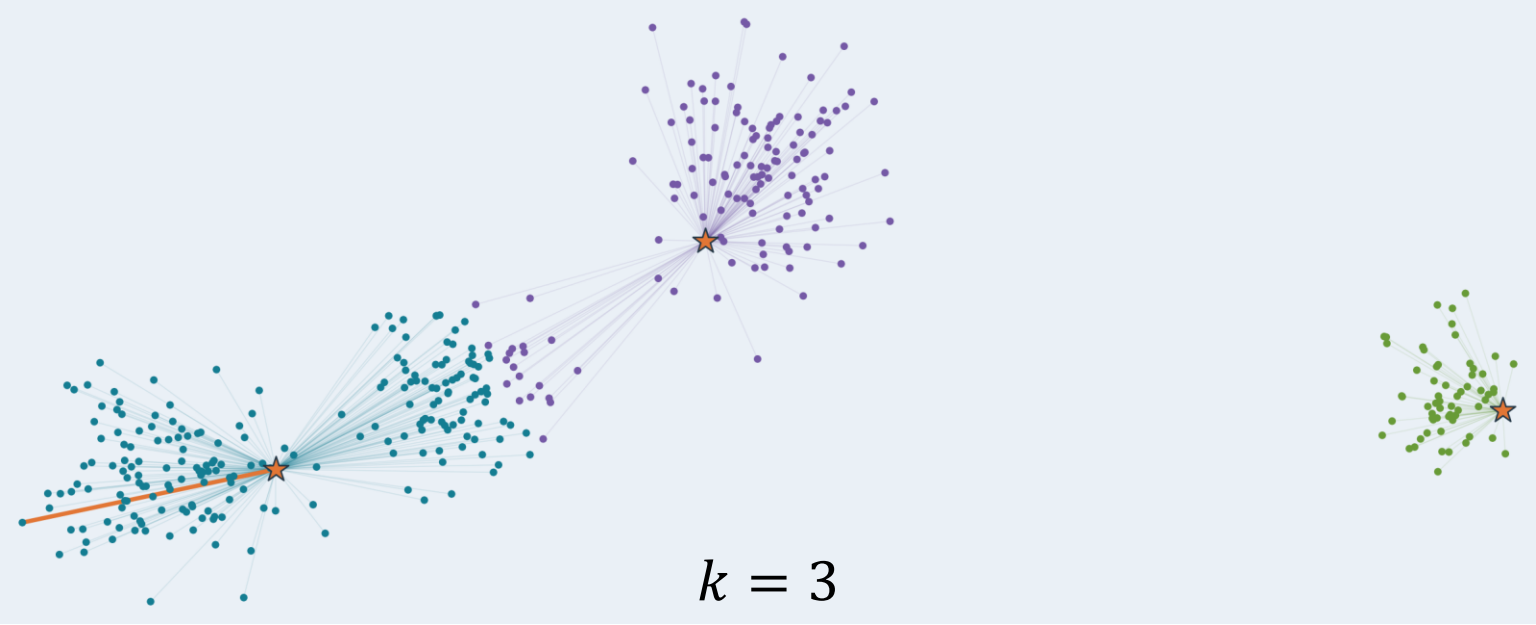


k-center

Select a subset of data points $Q \subseteq P$ such that

- Exactly k points are selected, i.e., $|Q| = k$
- The maximum distance to each point from the nearest selected point, i.e., $\max_{p_j \in P} \min_{p_i \in Q} d_{ij}$, is minimized

Nonlinear objective!



MILP for k-center

- $x_i = 1$ if point p_i is selected and 0 otherwise (a decision variable)
- $z_{ij} = 1$ if p_j is assigned to p_i and 0 otherwise (a decision variable)

$$\min$$

$$\text{s.t.} \sum_{i=1}^{n} x_i \leq k$$

At most k points are selected

$$\sum_{i=1}^{n} z_{ij} = 1, \quad j = 1, \ldots, n$$

Assigned to one selected point

$$z_{ij} \leq x_i, \quad i, j = 1, \ldots, n$$

Only assigned to a selected point

$$x_i, \in \{0,1\}, \quad i = 1, \ldots, n$$

$$z_{ij}, \in \{0,1\}, \quad i, j = 1, \ldots, n$$

MILP for k-center

- $x_i = 1$ if point p_i is selected and 0 otherwise (a decision variable)
- $z_{ij} = 1$ if p_j is assigned to p_i and 0 otherwise (a decision variable)
- C: the maximum distance (a continuous decision variable)

$$\min\ C$$

$$\text{s.t.}\ \sum_{i=1}^{n} x_i \le k$$

At most k points are selected

$$\sum_{i=1}^{n} z_{ij} = 1, \quad j = 1, \ldots, n$$

Assigned to one selected point

$$z_{ij} \le x_i, \quad i, j = 1, \ldots, n$$

Only assigned to a selected point

$$C \ge d_{ij} z_{ij}, \quad i, j = 1, \ldots, n$$

Link the maximum distance to individual distances

$$x_i, \in \{0,1\}, \quad i = 1, \ldots, n$$

$$z_{ij}, \in \{0,1\}, \quad i, j = 1, \ldots, n$$

MILP for k-center

- $x_i = 1$ if point p_i is selected and 0 otherwise (a decision variable)
- $z_{ij} = 1$ if p_j is assigned to p_i and 0 otherwise (a decision variable)
- C: the maximum distance (a continuous decision variable)

Minimization and the inequality constraints ensure that C matches the maximum distance from the nearest selected point

$$\min \ C$$

$$\text{s.t.} \sum_{i=1}^{n} x_i \leq k$$

At most k points are selected

$$\sum_{i=1}^{n} z_{ij} = 1, \quad j = 1, \ldots, n$$

Assigned to one selected point

$$z_{ij} \leq x_i, \quad i, j = 1, \ldots, n$$

Only assigned to a selected point

$$C \geq d_{ij} z_{ij}, \quad i, j = 1, \ldots, n$$

Link the maximum distance to individual distances

$$x_i, \in \{0,1\}, \quad i = 1, \ldots, n$$

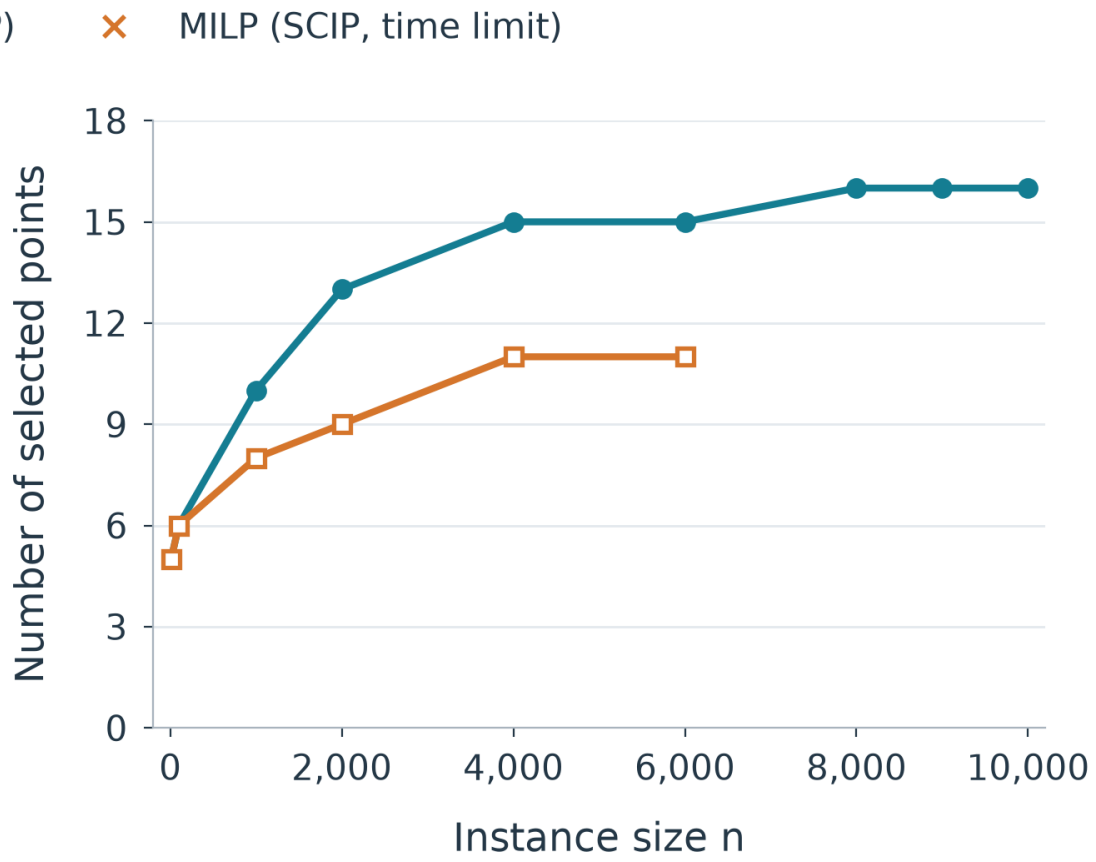
$$z_{ij}, \in \{0,1\}, \quad i, j = 1, \ldots, n$$

Experimental Evaluation

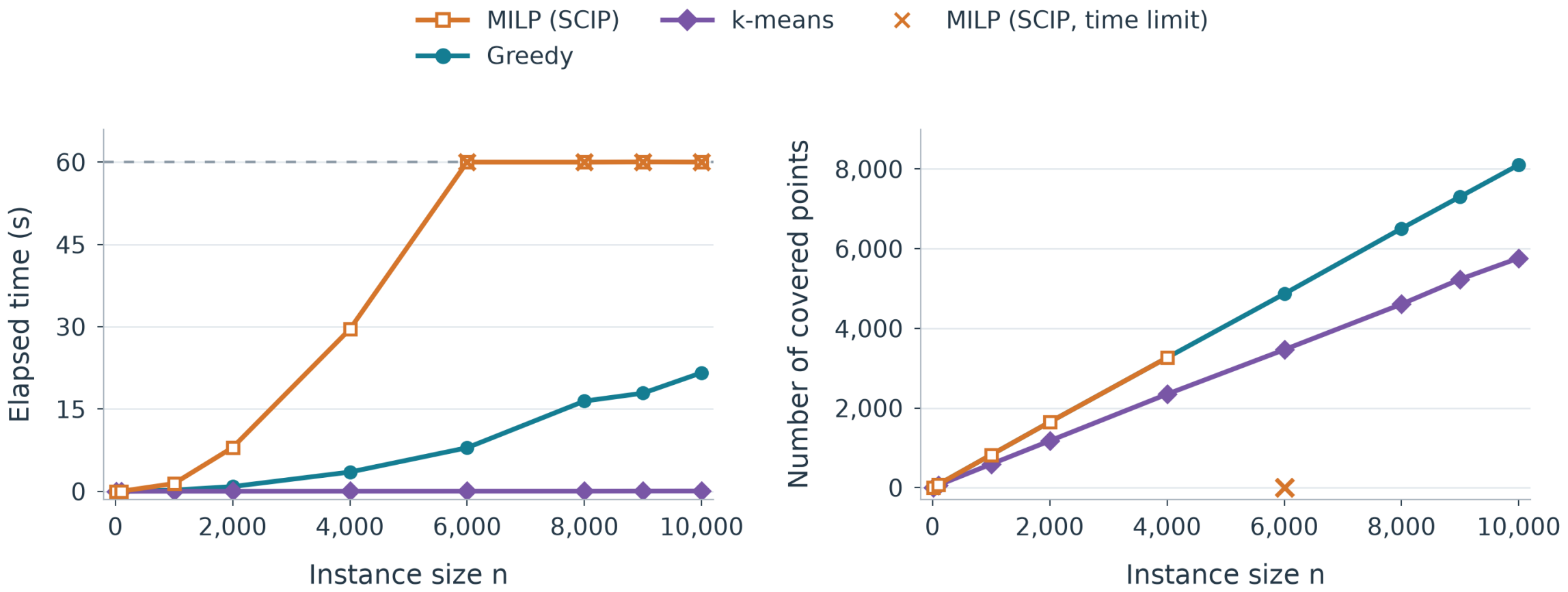
- Points in 2-dimensional Euclidean space, coordinates sampled from 4 different Gaussian distributions
- 1 instance for $n \in \{10,100,1000,2000,4000,6000,8000,9000,10000\}$
- $k = 3$ for maximum coverage, k-median, and k-center
- For k-center and k-median, we also run a greedy algorithm that selects the next point minimizing the objective value so far
- Greedy algorithms implemented in Python 3.12.3
- MILP models are implemented in Python with PySCIPOpt 6.3.0
- k-means and select centroids using scikit-learn 1.9.1
- 60-second time limit, 1 thread on my laptop (AMD Ryzen7 PRO 8840U)

Result for Set Covering

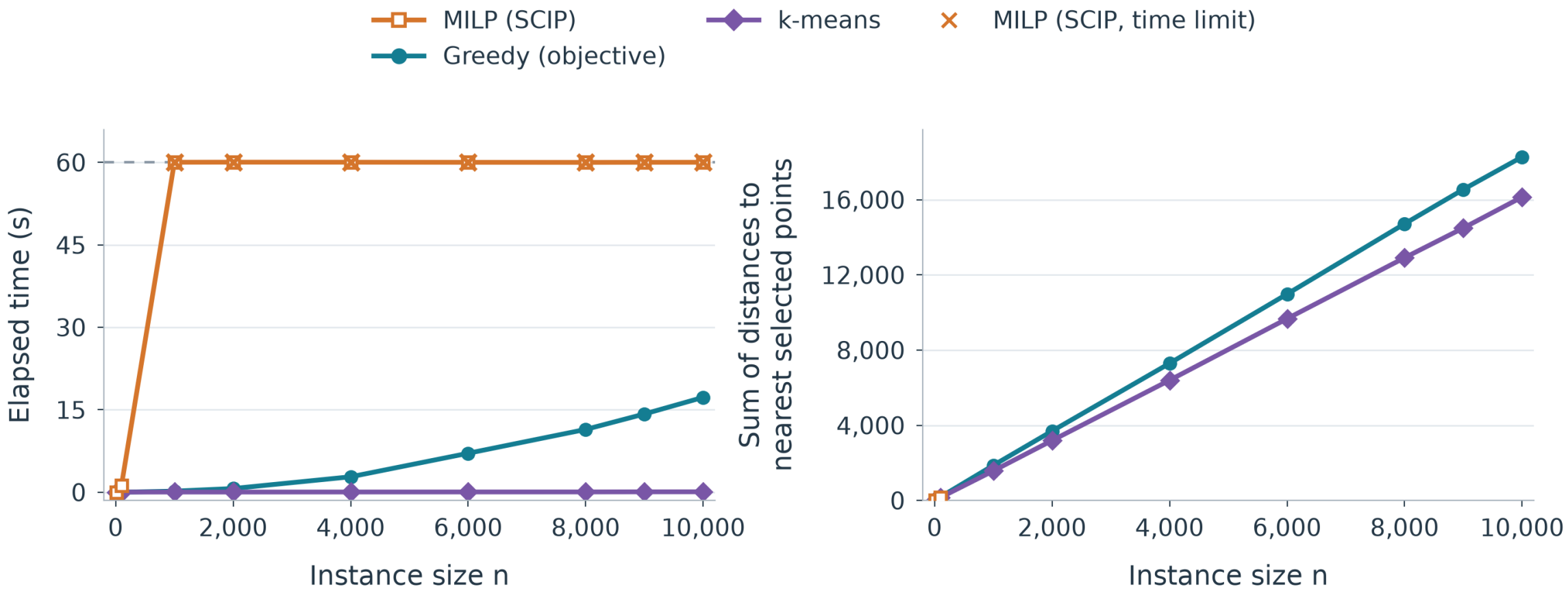




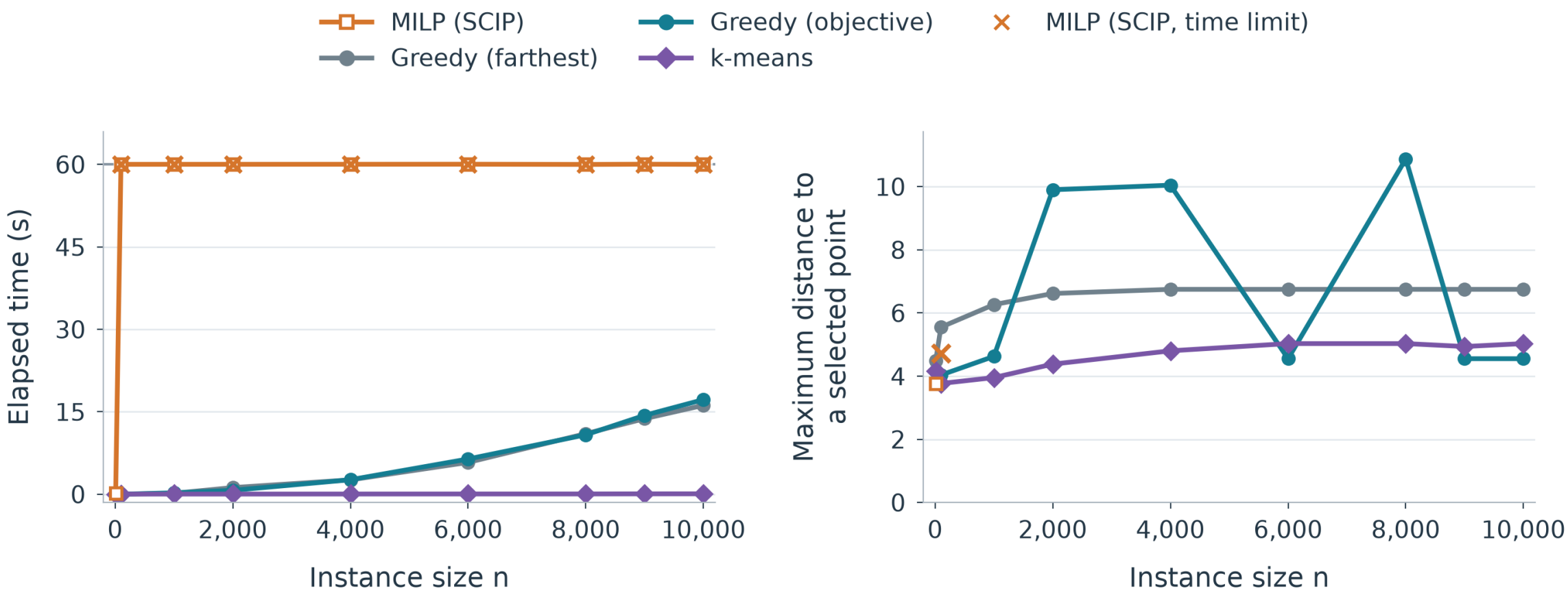
Result for Maximum Coverage



Result for k-median



Result for k-center



Summary of Results

- MILP works up to a certain size but fails in larger instances
- k-median and k-center are hard for MILP because of $O(n^2)$ decision variables and constraints
- In maximum coverage, greedy finds an optimal solution in practice, confirmed with up to $n = 4000$
- In k-center, greedy (objective) and k-means can be better than greedy (farthest) that has an approximation guarantee
- **Results could be different with different settings**, e.g., instance generation methods, parameters, random seeds, etc…

Beam Search: Beyond Greedy Selection

- Keep b best candidates in each layer ($b = 1$ matches greedy), e.g., maximize the number of covered points for set covering and maximum coverage
- Remove duplicate candidates

Beam Search: Beyond Greedy Selection

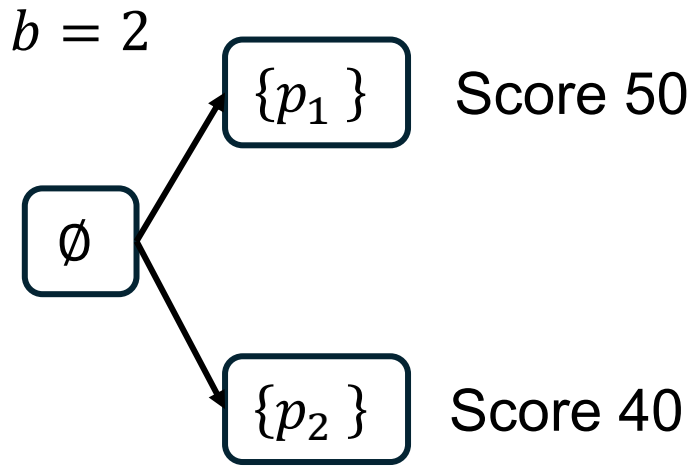
- Keep b best candidates in each layer ($b = 1$ matches greedy), e.g., maximize the number of covered points for set covering and maximum coverage
- Remove duplicate candidates

$b = 2$

$\emptyset$ Score 0

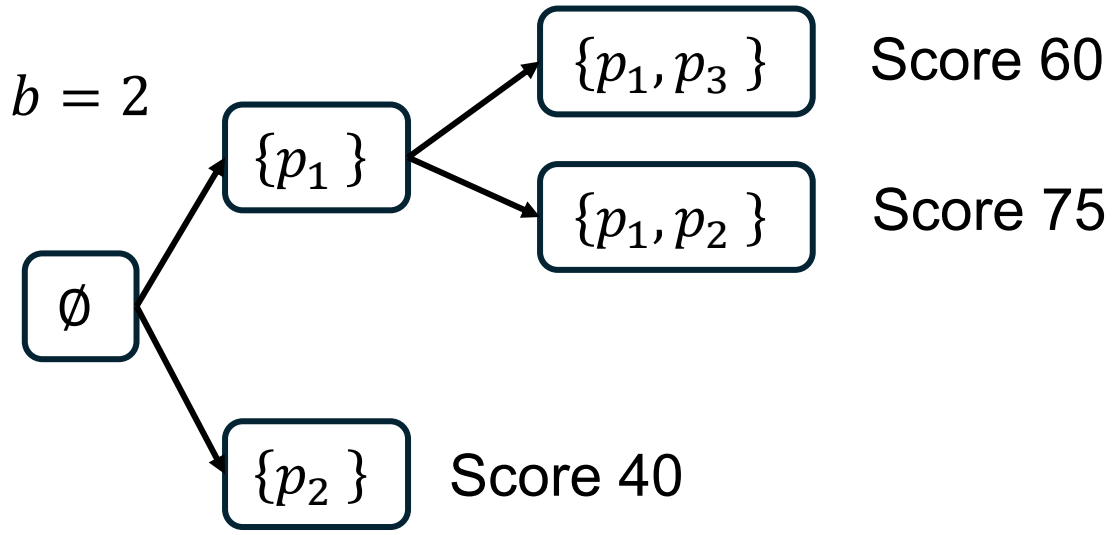
Beam Search: Beyond Greedy Selection

- Keep b best candidates in each layer ($b = 1$ matches greedy), e.g., maximize the number of covered points for set covering and maximum coverage
- Remove duplicate candidates



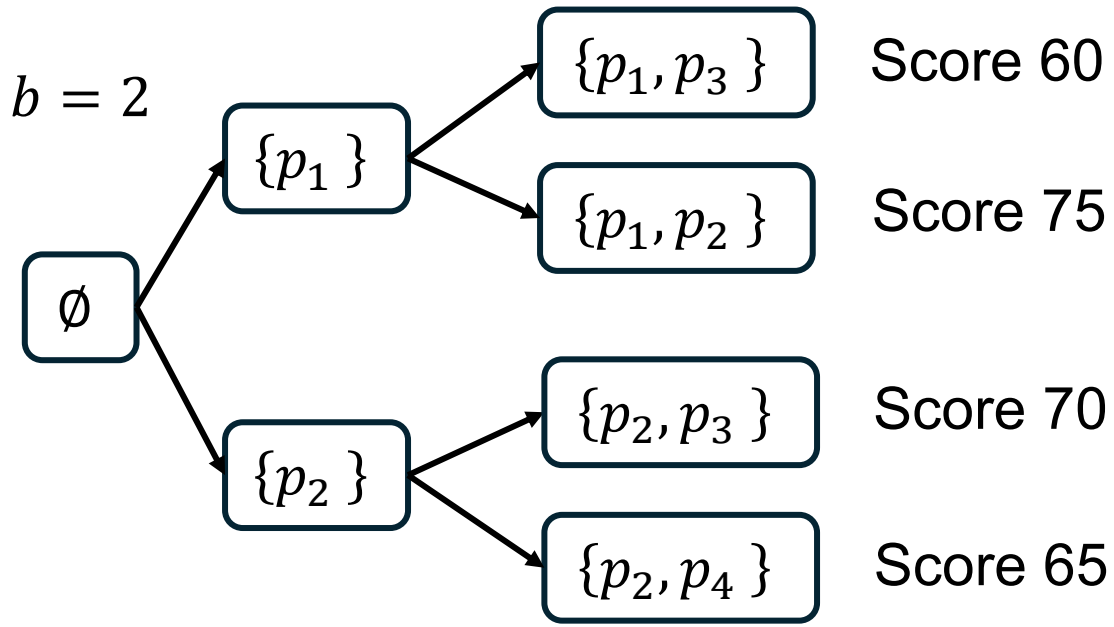
Beam Search: Beyond Greedy Selection

- Keep b best candidates in each layer ($b = 1$ matches greedy), e.g., maximize the number of covered points for set covering and maximum coverage
- Remove duplicate candidates



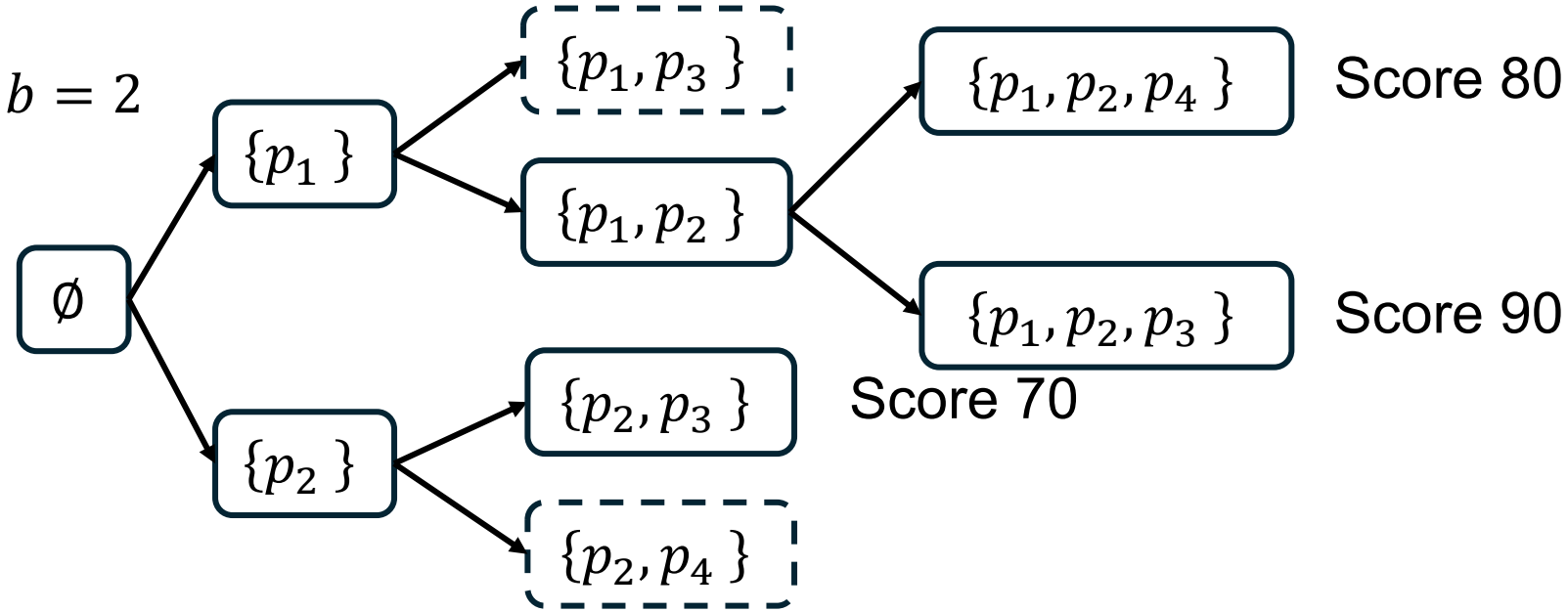
Beam Search: Beyond Greedy Selection

- Keep b best candidates in each layer ($b = 1$ matches greedy), e.g., maximize the number of covered points for set covering and maximum coverage
- Remove duplicate candidates



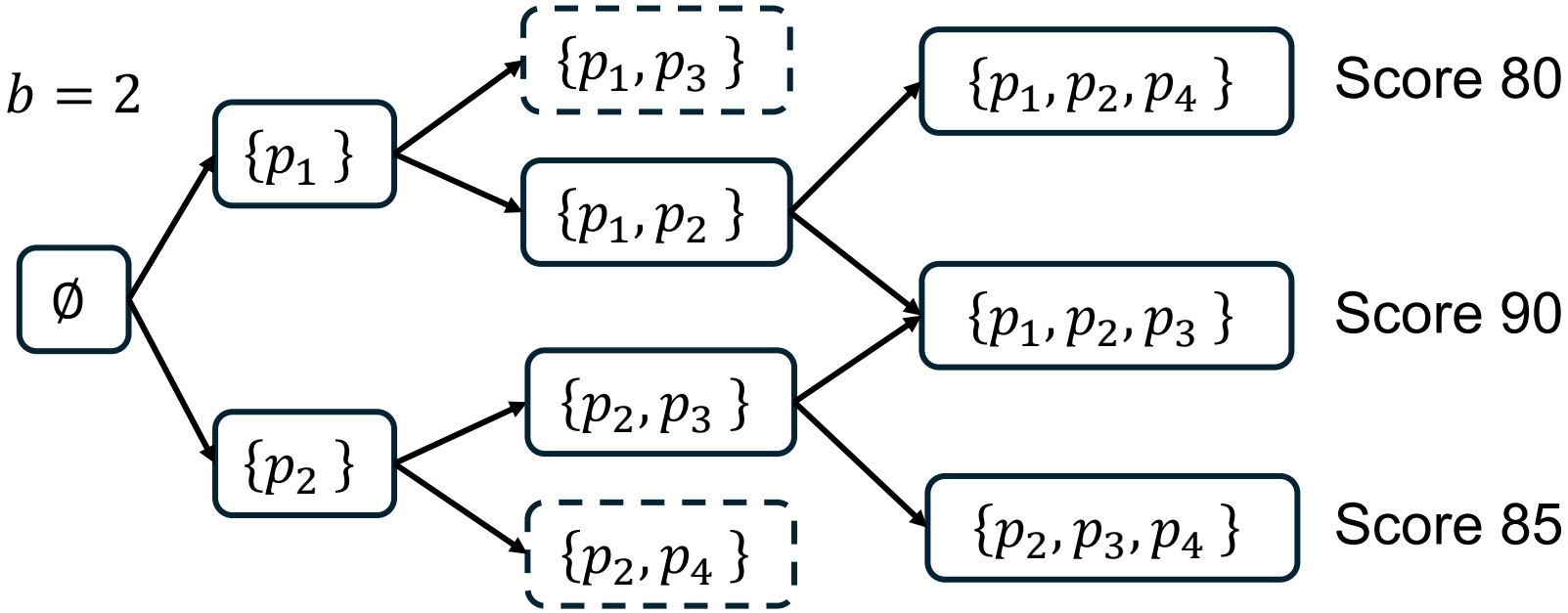
Beam Search: Beyond Greedy Selection

- Keep b best candidates in each layer ($b = 1$ matches greedy), e.g., maximize the number of covered points for set covering and maximum coverage
- Remove duplicate candidates



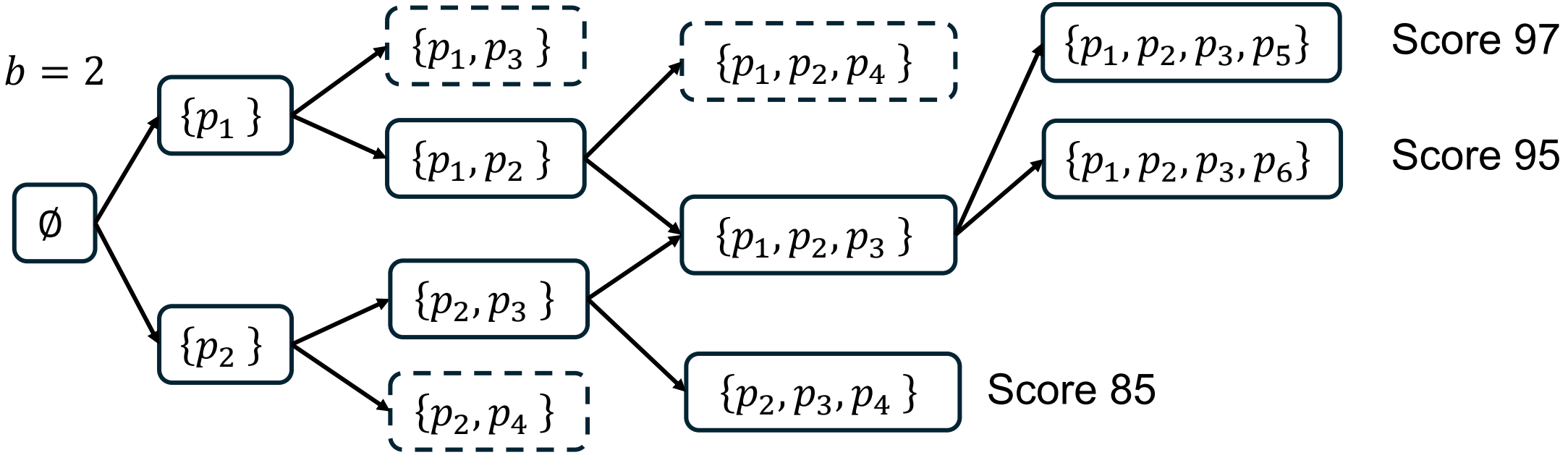
Beam Search: Beyond Greedy Selection

- Keep b best candidates in each layer ($b = 1$ matches greedy), e.g., maximize the number of covered points for set covering and maximum coverage
- Remove duplicate candidates



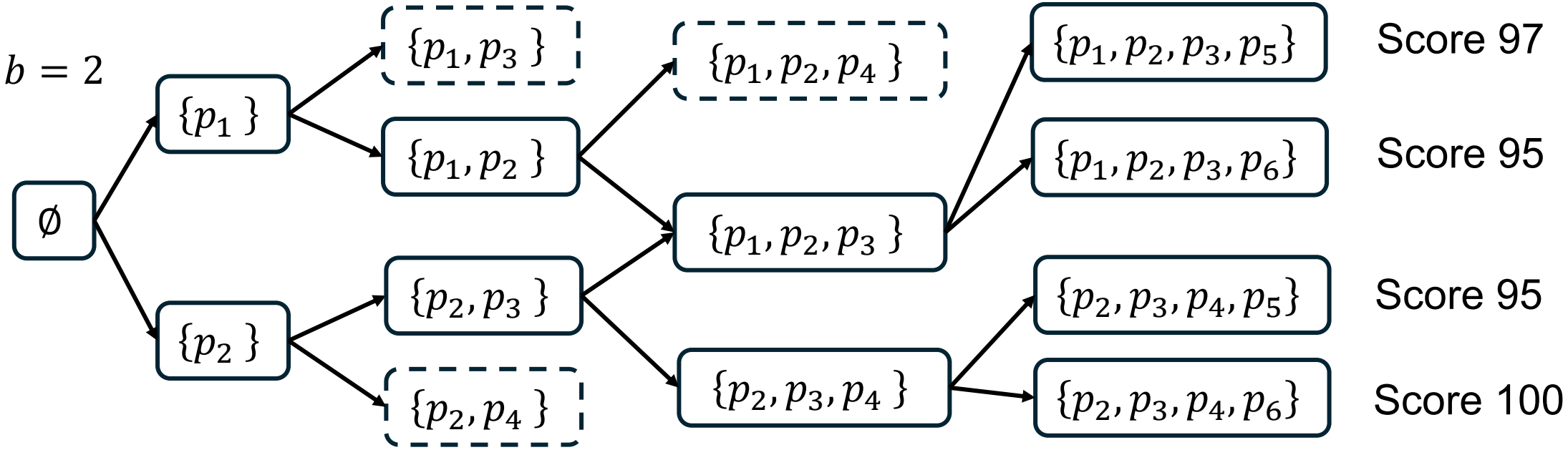
Beam Search: Beyond Greedy Selection

- Keep b best candidates in each layer ($b = 1$ matches greedy), e.g., maximize the number of covered points for set covering and maximum coverage
- Remove duplicate candidates



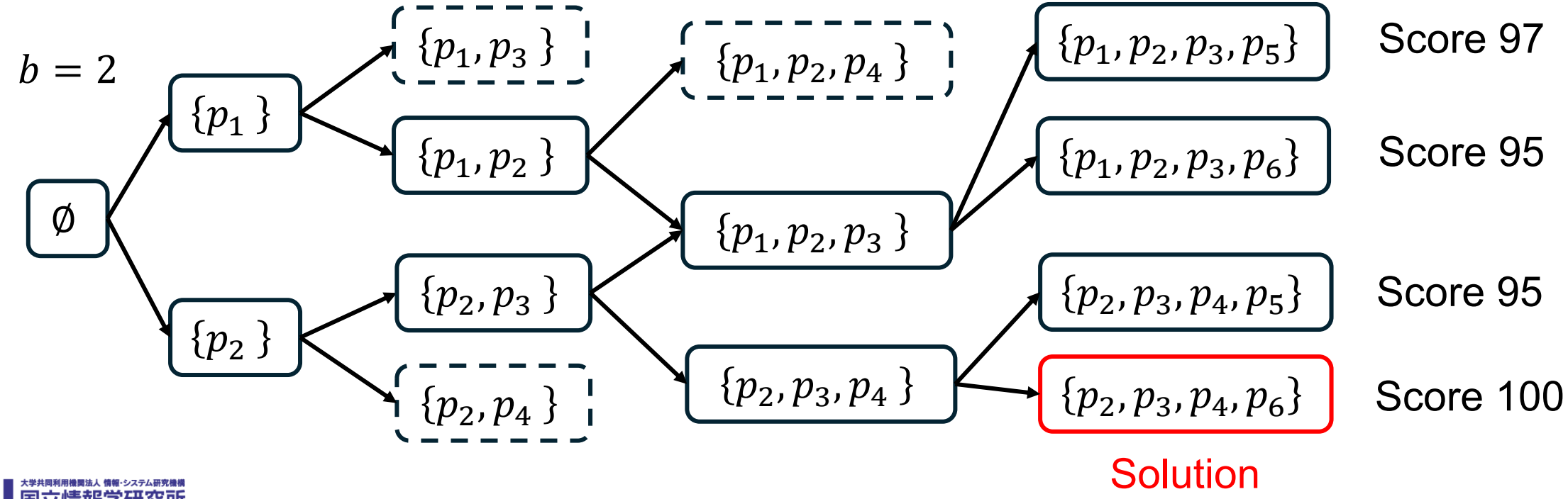
Beam Search: Beyond Greedy Selection

- Keep b best candidates in each layer ($b = 1$ matches greedy), e.g., maximize the number of covered points for set covering and maximum coverage
- Remove duplicate candidates

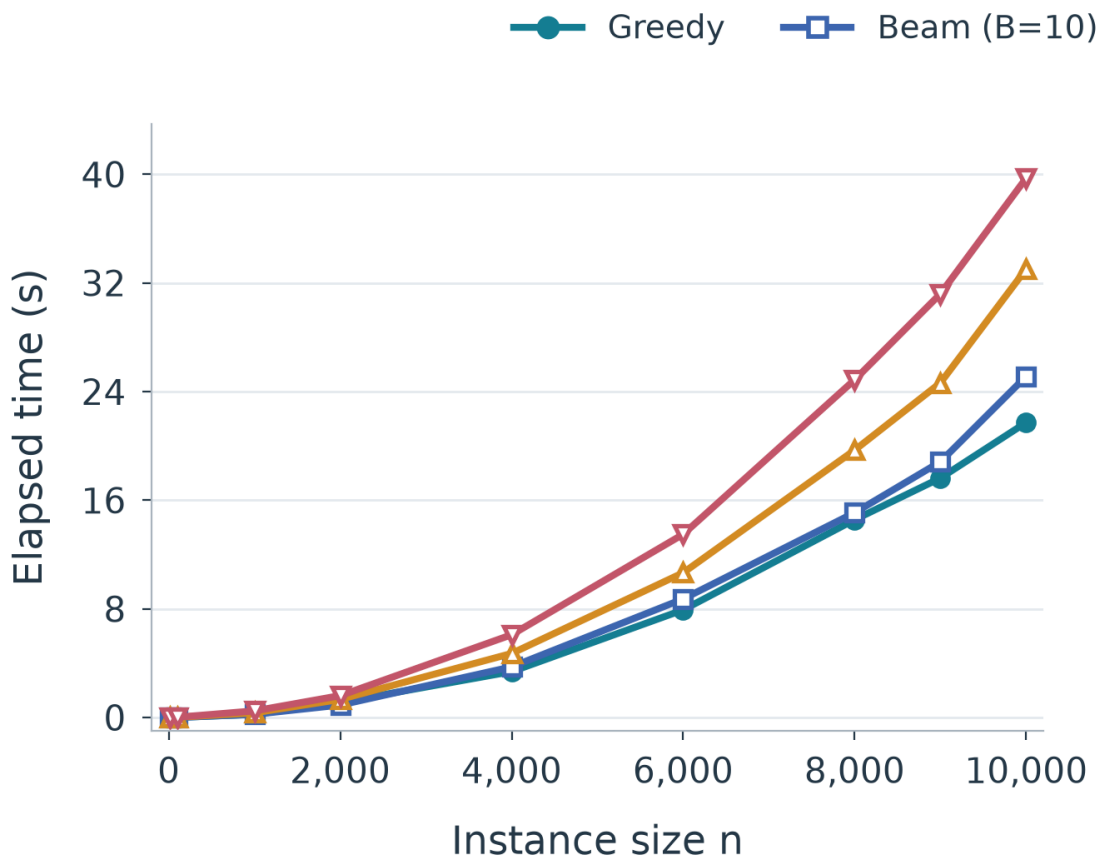


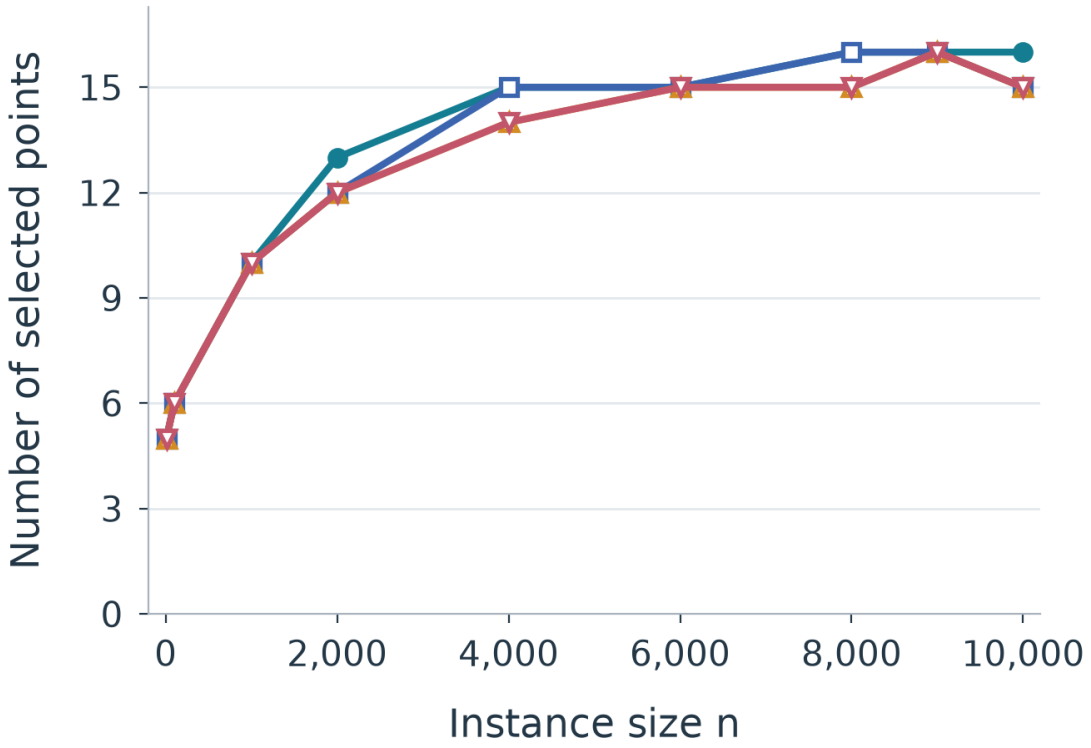
Beam Search: Beyond Greedy Selection

- Keep b best candidates in each layer ($b = 1$ matches greedy), e.g., maximize the number of covered points for set covering and maximum coverage
- Remove duplicate candidates

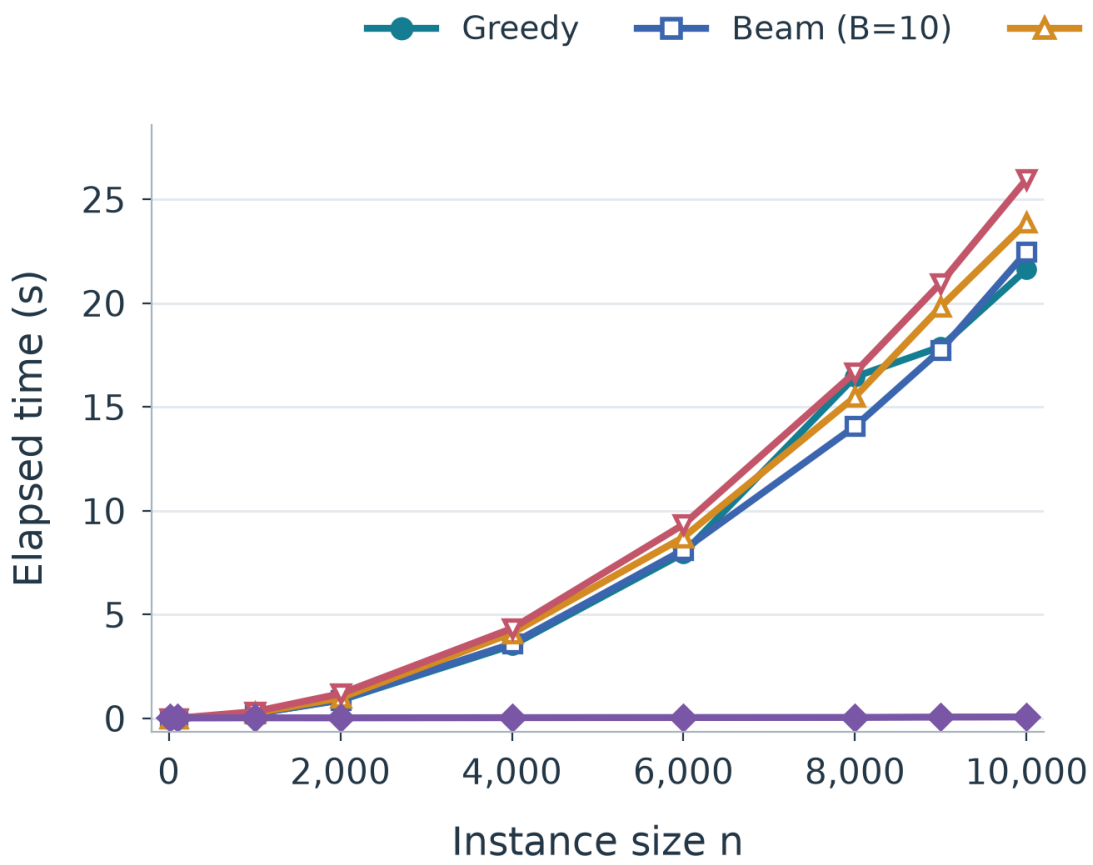


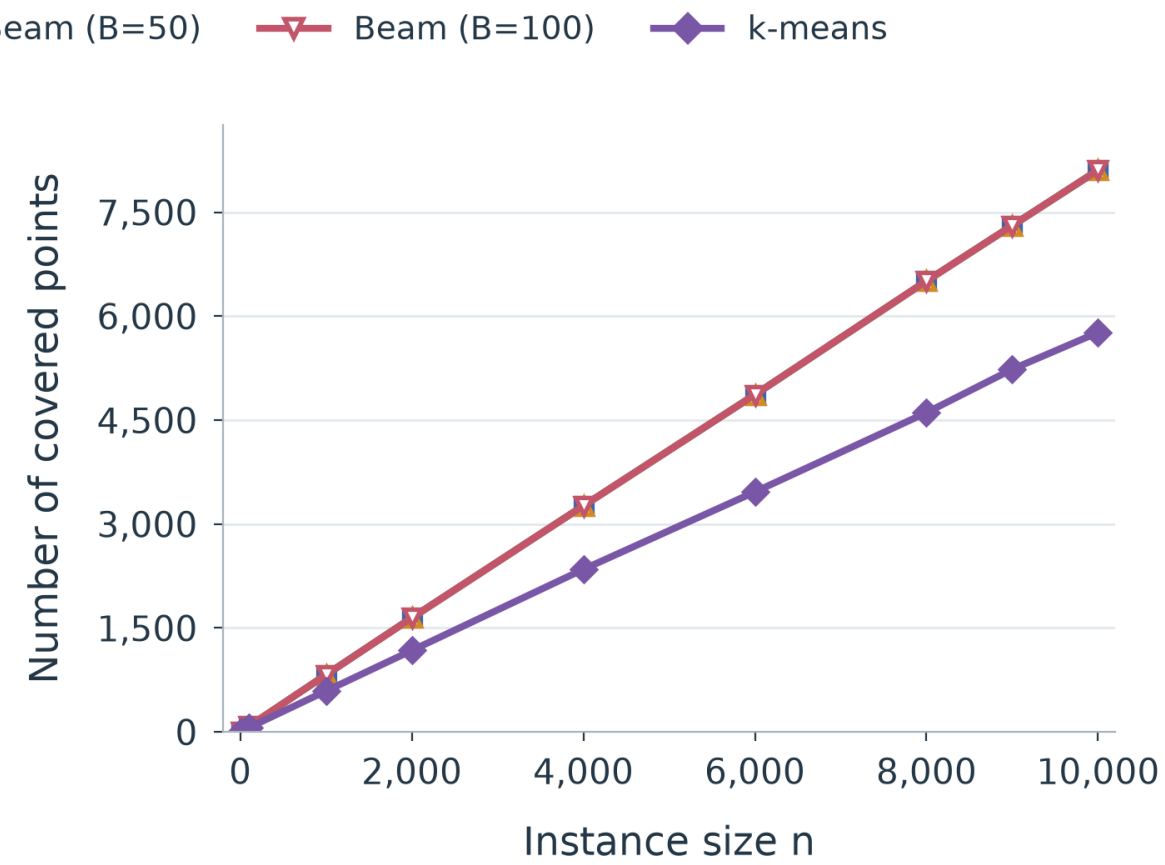
Beam Search for Set Covering



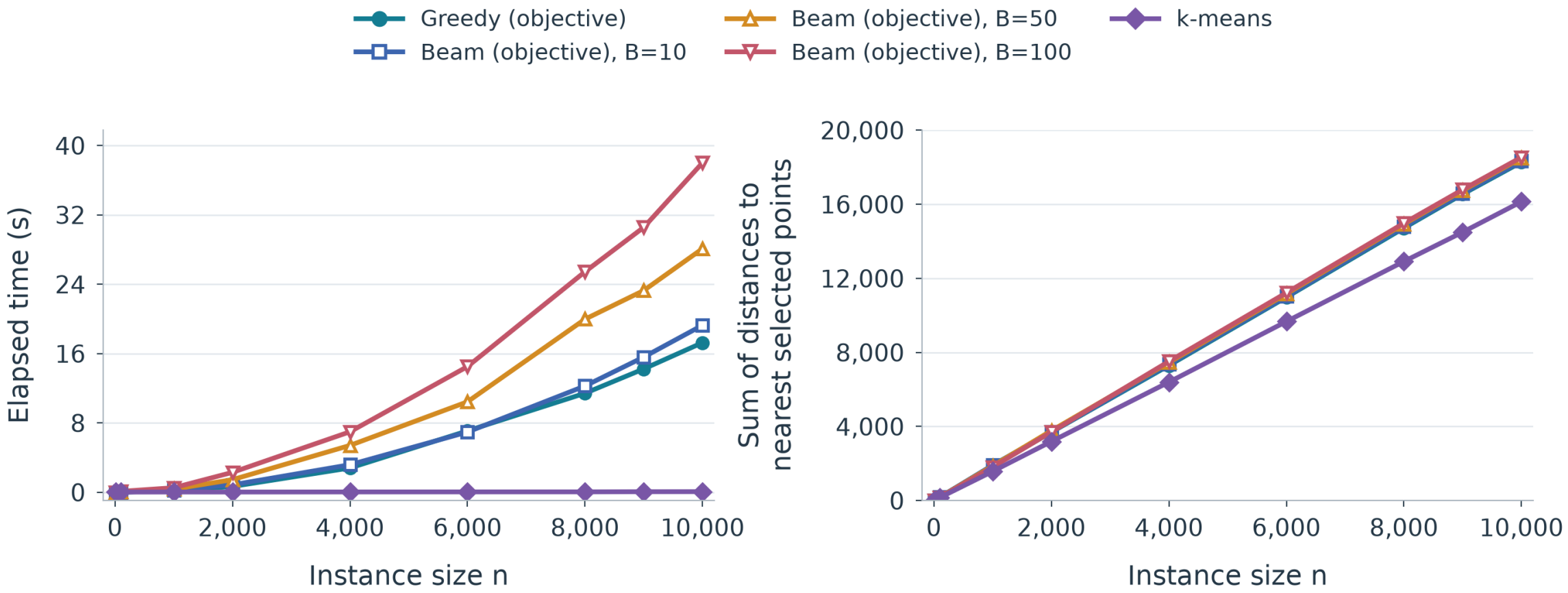


Beam Search for Maximum Coverage

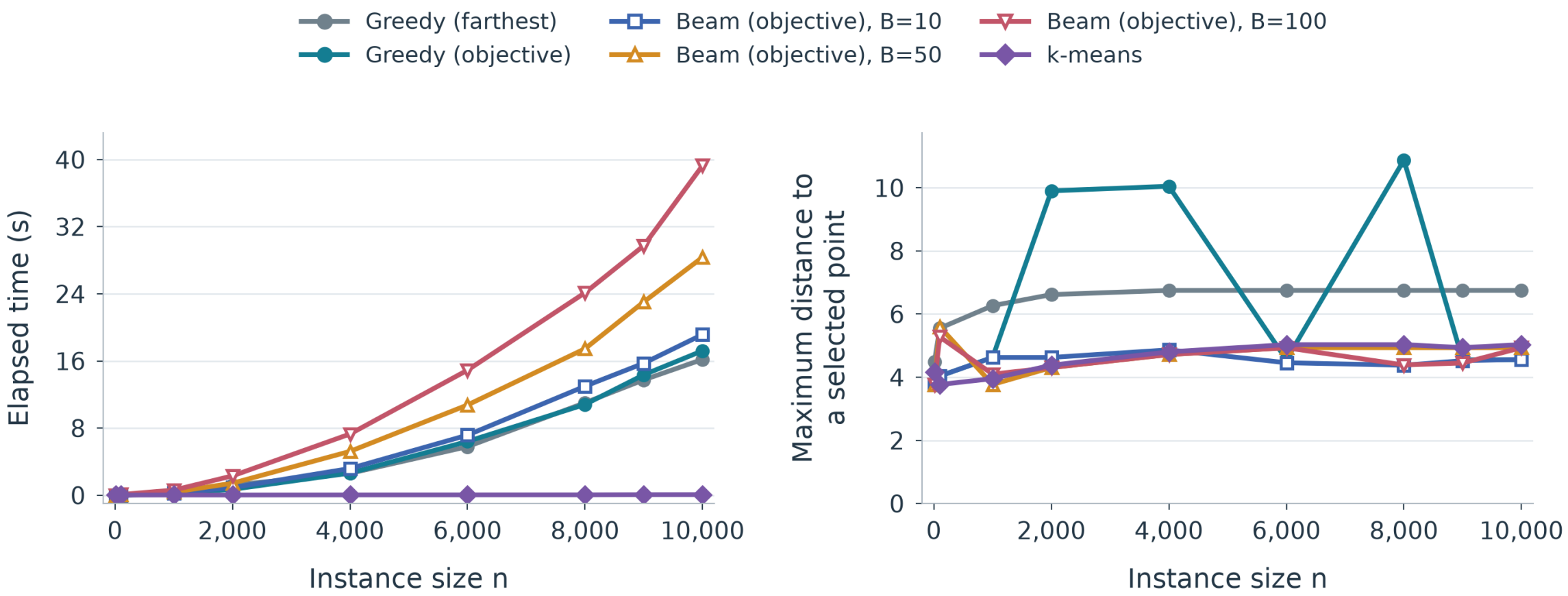




Beam Search for k-median



Beam Search for k-center



Summary of Beam Search Results

- Improvement in set covering and k-center
- No improvement in maximum coverage and k-median
- Increasing beam width b can result in a worse objective value
- Time does not increase linearly: $O(n^2)$ preprocessing takes time

Takeaways

- **Define your problem before solving it!**
- Combinatorial optimization can be useful for your problem
- Problem definition matters. Translate what you want to do into math
- **NP-hardness is not the end of the world**; you can try greedy algorithms (sometimes with an approximation guarantee), beam search, or general-purpose solvers

Topics not covered

- Different types of combinatorial optimization problems, e.g., the traveling salesperson problem, scheduling, packing
- Different types of algorithms, e.g., local search, metaheuristics
- Different solvers, e.g., constraint programming, dynamic programming

References

- Karp, Reducibility among combinatorial problems, *Complexity of Computer Computations*, 85-103, 1972.
- Papadimitrou, Worst-case and probabilistic analysis of a geometric location problem, *SIAM Journal on Computing* 10(3), 542-557, 1981.
- Clark, Colbourn, and Johnson, Unit disk graphs, *Discrete Mathematics* 86, 165-177, 1990.
- Chvatal, A Greedy heuristic for the set-covering problem, Mathematics of Operations Research 4(3), 233-255, 1979.
- Nemhauser, Wolsey, and Fisher, An analysis of approximations for maximizing submodular set functions—I, *Mathematical Programming* 14, 265-294, 1978.
- Gonzalez, Clustering to minimize the maximum intercluster distance, *Theoretical Computer Science* 38, 293-306, 1985.